

國立臺灣大學工學院工業工程學研究所



碩士論文

Department of Industrial Engineering

College of Engineering

National Taiwan University

Master's Thesis

區域尺度 StormCast 擴散模型之少步驟蒸餾：應用於臺灣對流尺度天氣預報

Few-Step Distillation of a Regional StormCast Diffusion
Model
for Taiwan Convective-Scale Weather Forecasting

Desmond Cheong

Desmond Cheong

指導教授: Advisor Name 博士

Advisor: Advisor Name, Ph.D.

中華民國 115 年 4 月

April 2026

國立臺灣大學碩士學位論文

口試委員會審定書



區域尺度 StormCast 擴散模型之少步驟蒸餾：應
用於臺灣對流尺度天氣預報

Few-Step Distillation of a Regional StormCast
Diffusion Model
for Taiwan Convective-Scale Weather Forecasting

本論文係 Desmond Cheong 君 (R00000000) 在國立臺灣大學工業工程學研究所完成之碩士學位論文，於民國 115 年 4 月 19 日承下列考試委員審查通過及口試及格，特此證明

口試委員：_____

(指導教授)

_____	_____
_____	_____
_____	_____
_____	_____

所 長：_____





Acknowledgements

I would like to thank my advisor for guidance throughout this thesis, and the members of the lab for the many discussions that shaped this work. I am grateful to the developers of StormCast and PhysicsNeMo for releasing the teacher checkpoints and training infrastructure that made this study possible, and to the Taiwan Central Weather Administration / NCDR for the RWRF reanalysis data used as training targets. Finally, I thank my family for their support throughout the program.





摘要

近年來，基於擴散模型的生成式天氣預報（例如 GenCast 與 StormCast）已展現出在機率性、銳利化、且可集成的短時預報任務上超越傳統確定性深度學習模型的能力。然而，這類模型在推論階段需要執行數十次神經網路前向傳遞（Number of Function Evaluations, NFE），使其在進行長時間自迴歸滾動預測或大規模集成預報時的計算成本過高，難以部署於區域對流尺度的實際業務場景。

本論文以一個以臺灣區域（RWRP）資料集訓練的 StormCast 擴散教師模型為起點，探討如何透過擴散蒸餾技術，在不顯著損失預報品質的前提下，將推論所需的 NFE 從約 25 步壓縮至 1 至 8 步。我們實作並比較兩種主流蒸餾方法：漸進式蒸餾（Progressive Distillation, PD）與一致性蒸餾（Consistency Distillation, CD），並另外訓練一個以條件流匹配（Conditional Flow Matching）為基礎的 FlowCast 學生模型，做為無需教師、直接學習雜訊到資料直線 ODE 的對照基準。針對臺灣 RWRP 資料集僅有 4 個高解析度輸出通道的特性，我們設計逐通道加權的 Pseudo-Huber 損失以及針對稀疏、重尾降水通道 q_{pepre} 的徑向功率譜密度正則化項。

我們在 2022 年整年逐小時驗證資料上，以確定性誤差（RMSE/MAE）、降水類別分數（CSI、FSS、不同閾值下的頻率偏差）、徑向功率譜，以及 1/3/6/12 小時滾動預測誤差等指標，系統性地比較學生模型與教師模型的表現。實驗結果顯示，

兩種蒸餾方法皆能在 4 步以內達到接近教師的確定性誤差，而一致性蒸餾在極端情況（1 步推論）下對降水銳利度的保持較為穩定。本研究為區域對流尺度擴散預報模型的加速部署提供了一套可重現的蒸餾流程與評估基準。



關鍵字：擴散模型、知識蒸餾、一致性模型、漸進式蒸餾、對流尺度預報、StormCast



Abstract

Diffusion-based generative weather forecasters such as GenCast and StormCast have recently surpassed deterministic deep-learning baselines on probabilistic, sharp, and ensemble-capable short-range prediction tasks. However, these models require tens of neural-network function evaluations (NFE) per forecast step at inference time, which makes autoregressive rollouts and large ensembles computationally prohibitive for operational regional, convective-scale deployment.

This thesis takes a pretrained StormCast EDM diffusion teacher trained on a Taiwan-domain RWRF dataset as its starting point, and investigates how diffusion distillation techniques can compress the sampling budget from roughly 25 NFE to between 1 and 8 NFE without sacrificing forecast quality. We implement and compare two mainstream distillation methods — Progressive Distillation (PD) and Consistency Distillation (CD) — and a third, teacher-free alternative based on Conditional Flow Matching, which we call FlowCast and which learns a straight-line noise-to-data ODE directly from the data. Be-

cause the Taiwan RWRf target contains only four high-resolution output channels ($t2m$, $u10$, $v10$, $qpepre$), we can afford channel-specific loss design: we use a channel-weighted Pseudo-Huber loss and add a radial power-spectral-density regularizer on the sparse, heavy-tailed precipitation channel $qpepre$.

We evaluate the distilled students against the teacher on the full 2022 hourly validation year using a precipitation-aware metric suite: per-channel RMSE/MAE, categorical precipitation scores (CSI, FSS, frequency bias) at operational thresholds, radial log-PSD, and 1/3/6/12-hour rollout errors. Our experiments show that both methods approach teacher-level deterministic error within four inference steps, and that consistency distillation preserves precipitation sharpness more robustly in the one-step regime. The thesis contributes a reproducible distillation pipeline and an evaluation protocol for accelerating regional convective-scale diffusion forecasters.

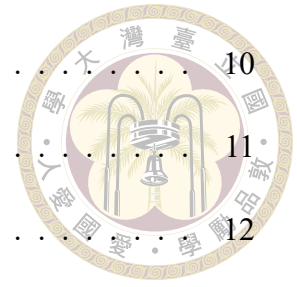
Keywords: Diffusion Models, Knowledge Distillation, Consistency Models, Progressive Distillation, Convective-Scale Forecasting, StormCast



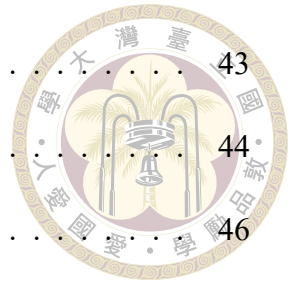
Contents

	Page
Verification Letter from the Oral Examination Committee	i
Acknowledgements	iii
摘要	v
Abstract	vii
Contents	ix
List of Figures	xiii
List of Tables	xv
Denotation	xvii
Chapter 1 Introduction	1
1.1 Weather Forecasting and Its Societal Stakes	1
1.2 A Brief History of Weather Prediction Methods	2
1.3 Diffusion Models for Weather: Intuition	4
1.4 The Inference-Cost Problem	4
1.5 Contributions	5
1.6 Thesis Outline	7
Chapter 2 Background and Related Work	9
2.1 Numerical Weather Prediction	9

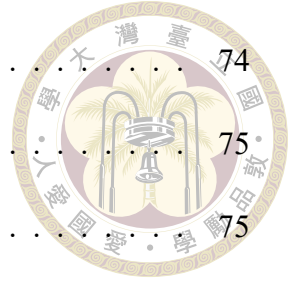
2.2	Data-Driven Global Weather Models	10
2.3	Regional and Convective-Scale Forecasting	11
2.4	GenCast and the Generative Turn	12
2.5	Score-Based Diffusion: A Primer	13
2.5.1	DDPM: The Discrete-Time Variance-Preserving Construction	14
2.5.2	DDIM: Non-Markovian Forward and Deterministic Sampling	16
2.5.3	Continuous-Time View: VE-SDE and PF-ODE	17
2.6	EDM: The Elucidated Formulation	18
2.7	Conditioning in Diffusion Models	20
2.8	Flow Matching and FlowCast	21
2.9	Why Distill?	23
2.10	Progressive Distillation	24
2.11	Consistency Models and Consistency Distillation	25
2.12	Adversarial Diffusion Distillation (Future Work)	27
2.13	Prior Applications of Distillation to Weather	27
2.14	Evaluation of Precipitation Forecasts	28
Chapter 3	Method	31
3.1	Problem Setup	31
3.2	Dataset: Taiwan RWRP	32
3.3	Data Sources and Preprocessing Pipeline	33
3.3.1	Raw data sources	33
3.3.2	Preprocessing pipeline	34
3.3.3	Target Distributions: Sparsity, Gaussianity, and the Diffusion Prior	39



3.4	Teacher Architecture Recap	43
3.5	Method A: Progressive Distillation	44
3.6	Method B: Consistency Distillation	46
3.7	Method C: FlowCast (Conditional Flow Matching)	47
3.8	Loss Design for the Precipitation Channel	53
3.9	Training Infrastructure	54
Chapter 4	Experiments and Results	57
4.1	Experimental Protocol	57
4.2	Evaluation Metrics	58
4.3	Progressive-Distillation Results	59
4.4	Consistency-Distillation Results	60
4.5	PD vs CD at Matched NFE	61
4.6	Qualitative Case Studies	61
4.7	Failure Modes	62
4.8	Discussion	63
Chapter 5	Conclusion and Future Work	65
5.1	Summary of Contributions	65
5.2	Limitations	66
5.3	Future Work	66
5.4	Closing Remarks	67
	References	69
	Appendix A — Hyperparameters and Training Configuration	73
A.1	EDM Teacher Hyperparameters	73



A.2	Progressive Distillation	74
A.3	Consistency Distillation	75
A.4	FlowCast (Conditional Flow Matching)	75
A.5	Per-Channel Dataset Statistics	77
Appendix B — Reproducibility		79
B.1	Code Availability	79
B.2	Dataset and Checkpoints	79
B.3	Commands	80
Appendix C — Detailed Derivations		83
C.1	DDPM Forward-Process Marginals	83
C.2	DDPM Posterior $q(x_{t-1} x_t, x_0)$	85
C.3	DDPM Variational Bound (ELBO)	86
C.4	DDPM Simplified ϵ -Loss	87
C.5	Tweedie’s Formula: Score and Denoiser	89
C.6	Probability-Flow ODE from the VE-SDE	89
C.7	DDIM: Non-Markovian Forward and Deterministic Sampler	91
C.8	Equivalence of ϵ , x_0 , and v Parameterisations	93
C.9	EDM Preconditioning Derivation	93
C.10	Karras ρ -Spaced Noise Schedule	96
C.11	CFM Theorem: Conditional = Marginal Gradient	96
C.12	Independent CFM: Straight-Line Path and Target	98
C.13	FlowCast’s Standardised-Residual Adaptation	99





List of Figures

Figure 3.1	Per-channel Gaussianity diagnostics for HighRes targets	40
------------	---	----





List of Tables

Table 3.1	Raw data sources used for training. Low-resolution ERA5 provides synoptic conditioning; RWRF and QPEPRE together supply the convection-permitting target.	35
Table 3.2	Gaussianity statistics of HighRes target channels	40





Denotation

NWP	Numerical Weather Prediction
RWRF	Regional Weather Research and Forecasting (Taiwan domain reanalysis)
EDM	Elucidated Diffusion Model (Karras et al., 2022)
PF-ODE	Probability-Flow Ordinary Differential Equation
SDE	Stochastic Differential Equation
NFE	Number of Function Evaluations (neural-net forward passes per sample)
PD	Progressive Distillation
CD	Consistency Distillation
ADD	Adversarial Diffusion Distillation
EMA	Exponential Moving Average
PSD	Power Spectral Density

CSI	Critical Success Index
FSS	Fractions Skill Score
CRPS	Continuous Ranked Probability Score
X_t	High-resolution atmospheric state at time t (target). Channels: t_2m , u_{10} , v_{10} , q_{pepre} .
S_t	Low-resolution conditioning state at time t (24 channels).
I	Time-invariant fields (land–sea mask, orography).
F_ξ	Deterministic regression network producing mean $M_t = F_\xi(X_{t-1}, S_t, I)$.
M_t	Regression mean prediction.
R_t	Residual $R_t = X_t - M_t$ learned by the diffusion model.
c	Conditioning bundle $c = \text{concat}(X_{t-1}, S_t, M_t, I)$.
D_ψ	Frozen diffusion teacher (EDMPrecond SongUNet).
D_θ	Distilled diffusion student.
σ	EDM noise level.
$\sigma_{\min}, \sigma_{\max}$	Minimum and maximum noise levels (0.002 and 80).
σ_{data}	EDM data standard-deviation hyperparameter (0.5).
ρ	Karras schedule exponent (7).
$N, N(k)$	Number of discretization steps (fixed or adaptive at training iter k).



μ_k

EMA decay for the consistency target network at iter k .

c_h

Pseudo-Huber constant.

β_k

Per-channel loss weight.

$P(\kappa)$

Radial power spectral density at wavenumber κ .







Chapter 1 Introduction

1.1 Weather Forecasting and Its Societal Stakes

Accurate short-range weather forecasting is a public-safety service. In Taiwan, a small, densely populated island lying along the western Pacific typhoon track, the stakes are especially concentrated: a single landfalling typhoon can dump more than 1000 mm of rainfall on steep orography in twelve hours, trigger flash floods and landslides, and disrupt transportation, agriculture, and power. Between typhoons, Mei-yu frontal systems, afternoon convective storms, and orographically forced rainfall drive the remainder of the hydrological cycle. For decision-makers who must issue evacuation orders, schedule reservoir releases, or re-route air traffic, the relevant question is almost always probabilistic and local: how much rain, where, within the next six to twelve hours?

The operationally critical quantity — precipitation at convective scales — is also the hardest for weather models to predict. Precipitation fields are sparse (most pixels are dry), heavy-tailed (the few wet pixels span orders of magnitude), and strongly coupled to terrain and mesoscale dynamics that only emerge at kilometre-scale resolution. This combination means that any forecasting system which produces visually smooth outputs, or which regresses toward the mean, will systematically underpredict the very events that matter most.

1.2 A Brief History of Weather Prediction Methods



Modern weather forecasting has passed through four broad eras, each driven by a different scientific or computational advance.

Numerical Weather Prediction (NWP). Since Richardson's early twentieth-century vision and its post-war realisation by Charney, Phillips, and others, operational forecasting has been dominated by numerical integration of the primitive equations of atmospheric motion. State-of-the-art systems such as the ECMWF Integrated Forecasting System (IFS), the NCEP Global Forecast System (GFS), and regional convection-permitting models like WRF and its Taiwan variant RWRF discretise the atmosphere on a three-dimensional grid, solve the governing fluid-dynamics and thermodynamics equations, and close the unresolved sub-grid processes (convection, microphysics, boundary-layer turbulence, radiation) through physical parameterisations. Uncertainty is quantified by ensemble prediction systems that perturb initial conditions and model physics. NWP is physically consistent, well-understood, and remarkably skilful, but convection-permitting runs are computationally expensive, parameterisation error is non-trivial, and operational latency is a fundamental constraint.

Statistical and classical machine-learning post-processing. Long before deep learning, statistical techniques — model output statistics (MOS), analog methods, kriging, and more recently random forests and gradient-boosted trees — have been used to bias-correct and downscale NWP outputs. These approaches treat the NWP forecast as a feature vector and learn a local correction to each observation site. They set an important precedent:

that learned components can complement physical cores, and that the ultimate criterion is predictive skill at points of interest rather than global physical fidelity.



The deep-learning era. Starting around 2020, a new generation of purely data-driven global weather models began to rival and in some cases surpass operational NWP at synoptic scales. FourCastNet [14] used an Adaptive Fourier Neural Operator to produce six-hourly global forecasts at 0.25° resolution. Pangu-Weather [2] introduced a three-dimensional hierarchical transformer and demonstrated medium-range skill competitive with IFS. GraphCast [9] adopted an icosahedral-mesh graph neural network and achieved operational-quality ten-day forecasts at a fraction of the inference cost of IFS. At convective scales, MetNet [1, 4, 21] and the Deep Generative Model of Rainfall (DGMR) [16] pushed radar-based nowcasting from deterministic CNNs toward generative models capable of producing sharp, ensemble-like outputs. Most recently, StormCast [13] demonstrated that a convection-permitting diffusion model can learn a km-scale regional forecaster directly from reanalysis data.

The generative turn. A persistent problem of deterministic deep-learning forecasters is that, because they are trained with pixelwise regression losses, they produce fields that are systematically blurry and that underestimate extremes. Generative models resolve this by learning the distribution of plausible forecasts rather than its conditional mean. DGMR showed this qualitatively for radar nowcasting; GenCast [15] made the argument at global scale, demonstrating that a conditional diffusion model produces both sharper and better-calibrated ensemble forecasts than GraphCast. The lesson transfers directly to regional convective forecasting, and it motivates the teacher model at the centre of this thesis.

1.3 Diffusion Models for Weather: Intuition



A diffusion model is a generative model trained to reverse a gradual noising process.

Given a clean sample x_0 drawn from some data distribution — say, a high-resolution precipitation field over Taiwan — we progressively add Gaussian noise until after enough steps the result is indistinguishable from pure noise. A neural network is then trained to undo one step of this process: given a noised sample and the current noise level, predict the underlying clean signal (equivalently, the score of the noised distribution). At sampling time we start from pure Gaussian noise and apply the learned denoiser repeatedly until we recover a clean sample.

Three properties make this framework attractive for weather forecasting. First, it is naturally probabilistic: different noise seeds produce different but equally plausible samples, giving an ensemble for free. Second, the samples are sharp: because the model is never asked to average over outcomes, it does not regress toward the mean. Third, the denoiser can be conditioned on auxiliary inputs — the previous atmospheric state, coarse boundary fields, invariant land-surface information — which is exactly the interface a forecaster needs. GenCast at global scale and StormCast at regional scale both exploit these properties.

1.4 The Inference-Cost Problem

The cost of this flexibility is paid at inference time. High-quality diffusion sampling requires on the order of 25 neural-network forward passes per generated sample — the so-called number of function evaluations (NFE). For a single forecast step this is tolerable,

but operational use cases compound it. An autoregressive 24-hour rollout multiplies the cost by 24. A 50-member ensemble for probabilistic forecasting multiplies it again. A convection-permitting Taiwan-domain forecast with a 50-member ensemble over a 24-hour horizon therefore requires on the order of $50 \times 24 \times 25 = 30,000$ forward passes of a large U-Net — which places the entire class of generative forecasters outside the operational envelope of a regional forecasting centre.

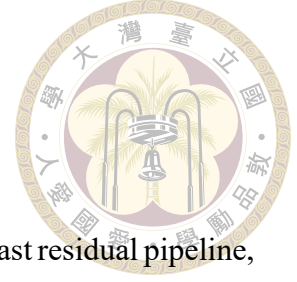
This thesis attacks the inference-cost problem directly, through diffusion distillation: given a trained teacher diffusion model, learn a student that approximates the teacher’s sampling trajectory in far fewer steps. The goal is to compress the 25-NFE teacher into a student that remains forecast-accurate at 1, 2, 4, or 8 NFE, thereby closing the gap between the best-available generative forecaster and what can actually be deployed.

1.5 Contributions

This thesis makes the following contributions:

1. We adapt Progressive Distillation [19] to the StormCast two-stage regression-plus-residual-diffusion architecture, distilling a 25-step EDM teacher down to 1 through 8 sampling steps on a four-channel Taiwan regional target.
2. We adapt Consistency Distillation [24] to the same setting, with a channel-weighted Pseudo-Huber loss and an auxiliary radial power-spectral-density regulariser on the sparse, heavy-tailed precipitation channel qpepre. The consistency target network uses an EMA schedule matched to the adaptive discretisation $N(k)$, and the teacher operator Φ is a Heun step rather than an Euler step, matching the sampler the EDM

teacher was trained to emulate.



3. We adapt Conditional Flow Matching [10, 17, 26] to the StormCast residual pipeline, producing a FlowCast student that is trained from scratch without a teacher and that, by learning a straight-line noise-to-data ODE, is competitive at one or two Euler steps. FlowCast is included as a direct one-training-run baseline against which the distilled students are measured.
4. We assemble a precipitation-aware evaluation suite — per-channel RMSE and MAE, categorical skill scores (CSI, FSS, frequency bias) at operational thresholds, radial log-PSD, and 1-, 3-, 6-, and 12-hour rollout errors — and run it on the full 2022 hourly validation year of the Taiwan RWRF dataset.
5. We present the first empirical three-way comparison on a regional convection-permitting diffusion forecaster between Progressive Distillation, Consistency Distillation, and a FlowCast baseline, with guidance on which method is preferred at which NFE budget.
6. Appendix B.3 provides full line-by-line derivations of DDPM, DDIM, VE-SDE / PF-ODE, EDM preconditioning, and Conditional Flow Matching — the theoretical machinery all four methods in this thesis rest on — in a form that should let a reader with a working knowledge of Gaussian algebra and the Fokker–Planck equation reconstruct every identity from scratch.

1.6 Thesis Outline



Chapter 2 surveys weather-prediction methods, diffusion theory, and diffusion distillation in enough depth to motivate and define every component used later. Chapter 3 describes the dataset, teacher architecture, and the two distillation pipelines as they are implemented in the accompanying codebase. Chapter 4 presents the experimental protocol and results across NFE budgets and rollout horizons. Chapter 5 summarises findings, limitations, and future work.





Chapter 2 Background and Related Work

This chapter assembles the theoretical background needed to understand the remainder of the thesis. It is organised in four parts: a survey of weather-prediction methods from NWP to data-driven models (Sections 2.1–2.4); the theory of score-based diffusion models, starting from the discrete-time DDPM construction, proceeding through DDIM and the continuous variance-exploding SDE / probability-flow ODE, and culminating in the EDM formulation used by our teacher (Sections 2.5–2.7); the theory of diffusion distillation, covering Progressive Distillation, Consistency Distillation, and Conditional Flow Matching as a one-training-run alternative (Sections 2.9–2.13); and an overview of precipitation-aware evaluation metrics (Section 2.14). Throughout the chapter we state the key identities in the main text and defer line-by-line derivations to Appendix B.3, which the reader may consult for any step marked “(see Appendix B.3)”.

2.1 Numerical Weather Prediction

Numerical weather prediction discretises the primitive equations of atmospheric motion — continuity, momentum (Navier–Stokes on a rotating sphere), thermodynamic en-

ergy, and water-vapour conservation — onto a three-dimensional grid and advances the state forward in time via explicit or semi-implicit integration. Sub-grid processes not captured by the dynamical core (convection, microphysics, boundary-layer turbulence, radiation, land-surface exchange) are closed through physical parameterisations. Operational systems include ECMWF's IFS (global spectral, 0.1° horizontal resolution), NCEP's GFS, and regional limited-area models such as WRF and its Taiwan-tuned variant RWRF, which provide the high-resolution targets used in this thesis.

Uncertainty is quantified by ensemble prediction systems that perturb initial conditions (e.g. via singular-vector or ensemble Kalman methods) and model physics. A convection-permitting (sub-4 km) ensemble with a few tens of members and a 24-hour forecast horizon requires hours of dedicated supercomputer time. The practical consequence is that NWP ensemble size and spatial resolution are traded off against operational latency.

2.2 Data-Driven Global Weather Models

Starting in 2022, a series of deep-learning models demonstrated that a single neural network, trained on the ERA5 reanalysis, can produce global medium-range forecasts that rival operational NWP on many headline metrics at a fraction of the inference cost.

FourCastNet. (author?) [14] used an Adaptive Fourier Neural Operator to produce six-hourly global forecasts at 0.25° resolution. It was the first model to demonstrate that a data-driven system could match IFS on synoptic variables within a plausible compute budget.

Pangu-Weather. (author?) [2] introduced a hierarchical three-dimensional transformer with a hierarchical time-aggregation scheme and demonstrated medium-range skill competitive with or exceeding IFS on several variables and lead times.



GraphCast. (author?) [9] adopted a multi-mesh graph neural network defined on an icosahedral discretisation of the sphere. It produces ten-day 0.25° forecasts that are competitive with IFS on the majority of evaluated fields and does so in roughly one minute on a single TPU, compared with hours on a supercomputer for NWP.

These models share an important limitation: because they are trained with deterministic pixelwise losses (typically latitude-weighted MSE), their forecasts become progressively blurrier with lead time and systematically underestimate extreme events. For convection and precipitation, which this thesis cares about most, this is disqualifying.

2.3 Regional and Convective-Scale Forecasting

Global models operate at resolutions too coarse to resolve convection and orographic rainfall over Taiwan. Three lines of work fill the gap.

Nowcasting models. MetNet-1/2/3 [1, 4, 21] produce high-resolution short-range precipitation forecasts by ingesting ground-based radar and satellite observations through large convolutional/attention backbones. DGMR [16] uses a conditional GAN to produce sharp, ensemble-like radar nowcasts and was the first work to make a strong empirical case that generative modelling is essential for precipitation skill.

StormCast. (author?) [13] introduced a convection-permitting regional diffusion fore-caster trained on the High-Resolution Rapid Refresh (HRRR) reanalysis over CONUS.

StormCast decomposes forecasting into two stages:

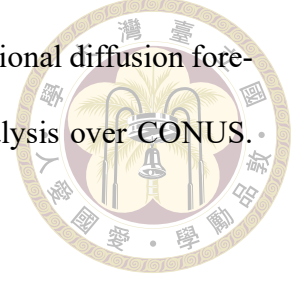
1. A deterministic regression network F_ξ (a StormCastUNet) takes the previous high-resolution state X_{t-1} , a low-resolution conditioning state S_t , and time-invariant fields I , and predicts the conditional mean $M_t = F_\xi(X_{t-1}, S_t, I)$.
2. A residual diffusion network D_ψ (an EDMPrecond SongUNet) models the residual $R_t = X_t - M_t$ conditionally on $c = \text{concat}(X_{t-1}, S_t, M_t, I)$.

At inference time the model is rolled out autoregressively, with the previous diffusion output feeding the next step's regression input. This decomposition has two payoffs: the deterministic network captures what is easy to predict with a single network call, leaving the diffusion model free to focus on the residual uncertainty; and the residual is closer to Gaussian than the raw fields, which stabilises diffusion training.

This thesis uses a Taiwan-adapted StormCast teacher trained on the RWRF dataset. The adaptation reduces the output state from the 99 channels of the CONUS original to just four: two-metre temperature (t2m), ten-metre zonal and meridional winds (u10, v10), and hourly precipitation (qpepre). This channel reduction has a useful side-effect: with only four output channels, per-channel loss engineering becomes cheap, and we exploit this in the distillation losses of Chapter 3.

2.4 GenCast and the Generative Turn

(author?) [15] introduced GenCast, a global 0.25° conditional diffusion forecaster



that samples ensemble members by varying the initial noise seed. GenCast was the first diffusion forecaster to clearly surpass the leading deterministic model (GraphCast) across the majority of its evaluation suite, on both deterministic scores (RMSE) and probabilistic scores (CRPS). Its success is the strongest empirical argument for the claim that motivates this thesis: generative modelling — and diffusion in particular — is the right inductive bias for short-to-medium range weather forecasting, but its operational cost is the obstacle to practical deployment.

2.5 Score-Based Diffusion: A Primer

A diffusion model defines a forward stochastic process that progressively corrupts a data sample $x_0 \sim p_{\text{data}}$ with Gaussian noise until the terminal state is indistinguishable from pure noise, then learns a neural network that reverses the corruption. The apparent difficulty of this programme — fitting the full generative reverse process of a high-dimensional distribution — hides a sequence of algebraic identities that together reduce training to a mean-squared-error regression. This section traces those identities in the order in which they were historically discovered: first the discrete-time variance-preserving DDPM construction of (author?) [6]; then the non-Markovian generalisation and deterministic sampler of DDIM [22]; then the continuous variance-exploding SDE and the probability-flow ODE that unify both [25]; and finally the elucidated EDM parameterisation used by our teacher [8]. Detailed line-by-line derivations of every equation in this section are collected in Appendix B.3.

2.5.1 DDPM: The Discrete-Time Variance-Preserving Construction



Forward process. DDPM defines a T -step Markov chain that adds Gaussian noise according to a variance schedule $\{\beta_t\}_{t=1}^T$:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I). \quad (2.1)$$

Setting $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$, induction (Appendix B.3, §C.1) gives the closed-form marginal

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I), \quad x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \quad (2.2)$$

This is the first key tractability result: the forward process never has to be simulated. For any training timestep t the corrupted sample x_t is drawn in one line using x_0 and a fresh Gaussian ϵ .

Tractable posterior. The reverse step $q(x_{t-1} | x_t)$ is intractable without x_0 , but the posterior conditioned on x_0 is Gaussian in closed form:

$$q(x_{t-1} | x_t, x_0) = \mathcal{N}(\tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I), \quad (2.3)$$

with

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\bar{\alpha}_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t, \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t. \quad (2.4)$$

The full derivation — combining Equations (2.1)–(2.2) with Bayes' rule and matching coefficients of a quadratic in x_{t-1} — is given in Appendix B.3, §C.2.

Variational bound. The reverse process $p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(\mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$ is trained to match q . The evidence lower bound (ELBO) on $\log p_\theta(x_0)$ decomposes into a sum of per-step KL divergences

$$-\log p_\theta(x_0) \leq \underbrace{D_{\text{KL}}(q(x_T \mid x_0) \parallel p(x_T))}_{L_T} + \sum_{t=2}^T \underbrace{D_{\text{KL}}(q(x_{t-1} \mid x_t, x_0) \parallel p_\theta(x_{t-1} \mid x_t))}_{L_{t-1}} - \underbrace{\log p_\theta(x_0 \mid x_1)}_{L_0}, \quad (2.5)$$

derived from Jensen's inequality on the joint $q(x_{1:T} \mid x_0)$ (Appendix B.3, §C.3). The noise-schedule design ensures L_T is approximately $D_{\text{KL}}(\mathcal{N}(0, I) \parallel \mathcal{N}(0, I)) = 0$ for large T ; L_0 is a discretised log-likelihood treated separately; the bulk of the loss is the middle sum, each term of which is a KL between two Gaussians with known variances.

Simplification to MSE. Because both sides are Gaussians with matched variances $\Sigma_\theta = \tilde{\beta}_t I$, the KL collapses to the squared distance of means:

$$L_{t-1} = \frac{1}{2\tilde{\beta}_t} \mathbb{E}_{x_0, \epsilon} [\|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|^2] + \text{const.} \quad (2.6)$$

Reparameterising μ_θ through a noise predictor ϵ_θ ,

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right), \quad (2.7)$$

and dropping the σ -dependent prefactor (empirically stable), yields the DDPM training loss used in practice:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2]. \quad (2.8)$$

Equation (2.8) is the punchline: the intractable log-likelihood has been replaced, without approximation on its upper bound and without approximation in the Gaussian-KL col-

lapse, by a supervised regression of the added noise. A line-by-line derivation that keeps every constant and justifies the dropped weighting is in Appendix B.3, §C.4.



2.5.2 DDIM: Non-Markovian Forward and Deterministic Sampling

DDPM’s sampler draws $T = 1000$ stochastic reverse steps, which is too slow for deployment. (author?) [22] observed that the DDPM training loss (2.8) depends on the marginals $q(x_t | x_0)$ but not on the particular Markov chain used to produce them. Any non-Markovian forward process with the same marginals yields the same trained network, but admits a different — potentially deterministic — sampler. DDIM parameterises such a family by

$$q_\sigma(x_{t-1} | x_t, x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_{t-1}} x_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \frac{x_t - \sqrt{\bar{\alpha}_t} x_0}{\sqrt{1 - \bar{\alpha}_t}}, \sigma_t^2 I\right), \quad (2.9)$$

parameterised by a free noise scale $\sigma_t \in [0, \tilde{\beta}_t]$. Setting $\sigma_t = 0$ yields a deterministic update

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0(x_t, t) + \sqrt{1 - \bar{\alpha}_{t-1}} \frac{x_t - \sqrt{\bar{\alpha}_t} \hat{x}_0(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}, \quad (2.10)$$

where $\hat{x}_0(x_t, t) = (x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)) / \sqrt{\bar{\alpha}_t}$ is the implied clean-data prediction. Equation (2.10) defines an integrator of an ODE: the same network, trained once, can be sampled with arbitrary sub-schedules and in far fewer steps (typically 20–50 NFE with comparable quality). A derivation in the equivalent variance-exploding form appears in Appendix B.3, §C.7. Because DDIM is the simplest first-order integrator of the underlying PF-ODE, it reappears as the building block of the distillation methods in Sections 2.10 and 2.11.

2.5.3 Continuous-Time View: VE-SDE and PF-ODE



(author?) [25] unified DDPM, noise-conditional score matching, and their continuous-time limits into a single SDE framework. In the variance-exploding (VE) formulation adopted by EDM and inherited by StormCast, the forward SDE is

$$dx = \sqrt{\frac{d\sigma^2(t)}{dt}} dW_t, \quad (2.11)$$

so that the closed-form marginal is the same one-line Gaussian corruption

$$x_\sigma = x_0 + \sigma \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad (2.12)$$

now indexed by a continuous noise level $\sigma \in [0, \sigma_{\max}]$. The remarkable structural fact is that every diffusion SDE admits a deterministic probability-flow ordinary differential equation (PF-ODE) whose marginals coincide with those of the SDE at every time:

$$\frac{dx}{d\sigma} = -\sigma \nabla_x \log p_\sigma(x), \quad (2.13)$$

where $\nabla_x \log p_\sigma(x)$ is the (Stein) score of the noised marginal. Equation (2.13) is derived in Appendix B.3, §C.6 by matching the Fokker–Planck equation of the SDE against the continuity equation of the deterministic flow. The PF-ODE plays a central role in distillation: it defines a deterministic trajectory $\sigma \mapsto x(\sigma)$ that links a pure-noise sample $x(\sigma_{\max})$ to its corresponding clean sample $x(\sigma_{\min})$, and any method that shortens the number of sampling steps is, one way or another, trying to approximate this trajectory with fewer function evaluations.

Score–denoiser equivalence (Tweedie). Training a diffusion model amounts to learning the score, typically indirectly through a denoiser $D(x_\sigma; \sigma)$ whose output predicts $\mathbb{E}[x_0 \mid x_\sigma]$. The score and the denoiser are related by Tweedie’s formula,

$$\nabla_x \log p_\sigma(x) = \frac{D(x; \sigma) - x}{\sigma^2}, \quad (2.14)$$

which we derive from the MMSE-denoiser identity in Appendix B.3, §C.5. The practical consequence is that denoising-score matching reduces to an ℓ_2 regression of the clean data from its noised version:

$$\mathcal{L}_{\text{DSM}}(\sigma) = \mathbb{E}_{x_0, \epsilon} [\|D_\theta(x_0 + \sigma\epsilon; \sigma) - x_0\|^2]. \quad (2.15)$$

Equation (2.15) is the continuous-time counterpart of Equation (2.8): the tractable training objective is a weighted mean-squared error over the noise scale, and the network can equivalently predict the clean data x_0 , the noise ϵ , or the $v = \alpha_t \epsilon - \sigma_t x_0$ target of (author?) [19]. The three parameterisations are interconvertible linearly and differ only in which pair of quantities the network sees as inputs and outputs; see Appendix B.3, §C.8.

2.6 EDM: The Elucidated Formulation

(author?) [8] proposed a unified parameterisation of diffusion training and sampling that cleans up a number of ad-hoc choices in earlier DDPM-style models. Our teacher, and by extension the students we distil, uses this formulation in full. The EDM preconditioning scalings are not heuristics: they are the unique one-parameter family that makes the denoiser’s input statistics, target statistics, and per- σ loss scale simultaneously uniform across σ . A line-by-line derivation of the $c_{\text{skip}}, c_{\text{out}}, c_{\text{in}}, c_{\text{noise}}, \lambda(\sigma)$ formulas below is given

in Appendix B.3, §C.9.



Noise schedule. EDM uses a ρ -spaced discretisation of noise levels between $\sigma_{\min}$ and $\sigma_{\max}$:

$$\sigma_i = \left(\sigma_{\max}^{1/\rho} + \frac{i}{N-1} (\sigma_{\min}^{1/\rho} - \sigma_{\max}^{1/\rho}) \right)^\rho, \quad (2.16)$$

for $i = 0, \dots, N-1$, with $\sigma_{\min} = 0.002$, $\sigma_{\max} = 80$, and $\rho = 7$. The schedule concentrates noise levels near $\sigma_{\min}$, which is where most of the useful signal is generated.

Preconditioning. A naive denoiser $F_\theta(x_\sigma, \sigma)$ must cope with inputs whose magnitudes vary by orders of magnitude. EDM addresses this by wrapping the inner network in skip, input, output, and noise scalings:

$$D_\theta(x_\sigma; \sigma) = c_{\text{skip}}(\sigma) x_\sigma + c_{\text{out}}(\sigma) F_\theta(c_{\text{in}}(\sigma) x_\sigma, c_{\text{noise}}(\sigma)), \quad (2.17)$$

where

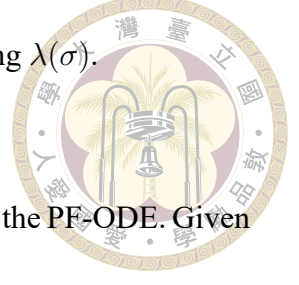
$$\begin{aligned} c_{\text{skip}}(\sigma) &= \sigma_{\text{data}}^2 / (\sigma^2 + \sigma_{\text{data}}^2), & c_{\text{out}}(\sigma) &= \sigma \cdot \sigma_{\text{data}} / \sqrt{\sigma^2 + \sigma_{\text{data}}^2}, \\ c_{\text{in}}(\sigma) &= 1 / \sqrt{\sigma^2 + \sigma_{\text{data}}^2}, & c_{\text{noise}}(\sigma) &= \frac{1}{4} \log \sigma. \end{aligned}$$

These scalings ensure that F_θ always sees inputs of unit variance and predicts an unnormalised residual, which is numerically stable across the full range of σ .

Training loss. EDM trains with a weighted denoising-score-matching loss:

$$\mathcal{L}_{\text{EDM}} = \mathbb{E}_{x_0, \sigma, \epsilon} \left[\lambda(\sigma) \|D_\theta(x_0 + \sigma\epsilon; \sigma) - x_0\|^2 \right], \quad (2.18)$$

with a log-normal distribution over σ and a Karras-specified weighting $\lambda(\sigma)$.



Heun sampler. EDM samples via a second-order Heun integrator of the PF-ODE. Given x_i at noise level σ_i , the next sample is

$$d_i = (x_i - D_\theta(x_i; \sigma_i))/\sigma_i, \quad (2.19)$$

$$x_{i+1}^{(\text{euler})} = x_i + (\sigma_{i+1} - \sigma_i) d_i, \quad (2.20)$$

$$d'_i = (x_{i+1}^{(\text{euler})} - D_\theta(x_{i+1}^{(\text{euler})}; \sigma_{i+1}))/\sigma_{i+1}, \quad (2.21)$$

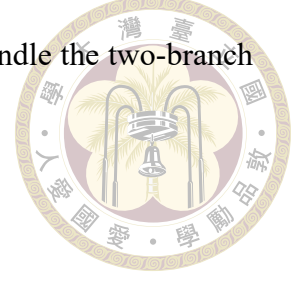
$$x_{i+1} = x_i + (\sigma_{i+1} - \sigma_i) \frac{1}{2}(d_i + d'_i). \quad (2.22)$$

Each step uses two denoiser evaluations except the final step where the correction is skipped. With $N = 18$ steps this yields roughly $2N - 1 = 35$ NFE; StormCast uses a shorter $N \approx 13$ schedule giving ≈ 25 NFE. Because the Heun step is what the teacher’s denoiser was trained to support, it must also be the teacher operator used by any distillation method that samples along the PF-ODE — a subtle but load-bearing point we return to in Section 2.11.

2.7 Conditioning in Diffusion Models

Conditional diffusion models can be implemented either by concatenating the condition c along the channel axis of the denoiser’s input, or by classifier-free guidance (CFG) [7], which trains the network jointly on conditional and unconditional tasks and linearly combines their outputs at inference time. StormCast uses the simpler concatenation approach: the conditioning bundle $c = \text{concat}(X_{t-1}, S_t, M_t, I)$ is stacked with the noised target along the channel dimension and fed to the SongUNet. No classifier-free guid-

ance is used, so the distillation methods we inherit do not need to handle the two-branch evaluation that CFG would require.



2.8 Flow Matching and FlowCast

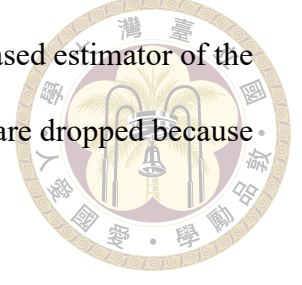
Conditional Flow Matching (CFM) is a recently-proposed alternative to score-based diffusion that learns the same kind of noise-to-data generative model — a deterministic flow integrated from a simple prior to the data distribution — but with a training objective that bypasses score matching entirely. Because the method is adopted as a third training route in this thesis (Section 3.7) we give it the same theoretical treatment as DDPM, DDIM, and EDM.

Continuous normalising flows, simulation-free training. A continuous normalising flow (CNF; 3) parameterises a deterministic velocity field $v_\theta(x, t)$, $t \in [0, 1]$, and pushes a prior sample $x_0 \sim \mathcal{N}(0, I)$ along the ODE $\dot{x}_t = v_\theta(x_t, t)$ to obtain x_1 , which we would like to distribute as p_{data} . Classical maximum-likelihood CNF training requires evaluating $\log p_\theta(x_1)$ via the Jacobian trace over the whole trajectory — a prohibitive cost at image or video scale. (author?) [10] and (author?) [26] instead train v_θ by direct regression against a target vector field $u_t(x \mid z)$ associated with a tractable conditional probability path $p_t(x \mid z)$ indexed by a latent z :

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t \sim \mathcal{U}(0,1), z \sim q(z), x \sim p_t(x|z)} \left[\|v_\theta(x, t) - u_t(x \mid z)\|^2 \right]. \quad (2.23)$$

The key theoretical result [10, Thm. 2] is that the gradient of (2.23) with respect to θ equals the gradient of the marginal flow-matching loss with the intractable marginal target field

— so training the network against a conditional target yields an unbiased estimator of the true field. A derivation of this identity, along with the constants that are dropped because they do not depend on θ , is given in Appendix B.3, §C.11.



Independent CFM (I-CFM) and the straight-line path. The simplest instantiation, Independent CFM [26], takes $z = (x_0, x_1)$ with $x_0 \sim \mathcal{N}(0, I)$ independent of $x_1 \sim p_{\text{data}}$, and defines the straight-line Gaussian probability path

$$p_t(x \mid x_0, x_1) = \mathcal{N}((1-t)x_0 + tx_1, \sigma_{\text{path}}^2 I), \quad x_t = (1-t)x_0 + tx_1 + \sigma_{\text{path}} \eta, \quad \eta \sim \mathcal{N}(0, I). \quad (2.24)$$

Differentiating the conditional mean with respect to t gives the corresponding target vector field

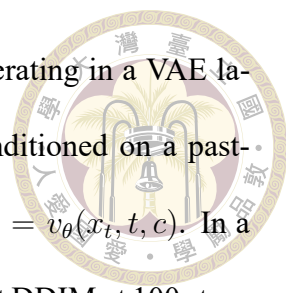
$$u_t(x \mid x_0, x_1) = x_1 - x_0, \quad (2.25)$$

which is independent of x — exactly the straight-line constant-velocity interpolation between the noise sample and the data sample. The training objective becomes

$$\mathcal{L}_{\text{I-CFM}} = \mathbb{E}_{t, x_0, x_1, \eta} \left[\|v_\theta((1-t)x_0 + tx_1 + \sigma_{\text{path}} \eta, t) - (x_1 - x_0)\|^2 \right]. \quad (2.26)$$

The path noise $\sigma_{\text{path}} > 0$ “thickens” the otherwise-singular conditional paths and regularises the vector-field fit, following (author?) [26]; FlowCast uses $\sigma_{\text{path}} = 0.01$. A derivation of (2.25) from (2.24) and an explicit comparison with the curved probability flow of DDPM are in Appendix B.3, §C.12.

FlowCast. (author?) [17] introduced FlowCast, the first fully probabilistic application of I-CFM to precipitation nowcasting. FlowCast trains a single U-Net backbone — in



the original paper an Earthformer-style cuboid-attention network operating in a VAE latent space — to regress the straight-line vector field $v_\theta(x_t, t, c)$ conditioned on a past-observation bundle c . Sampling uses a 10-step Euler integrator of $\dot{x}_t = v_\theta(x_t, t, c)$. In a direct ablation on the SEVIR benchmark, FlowCast at 1 ODE step beat DDIM at 100 steps on CRPS and CSI, and matched or exceeded the state-of-the-art probabilistic nowcasting baselines CasCast and PreDiff at a fraction of the inference cost. Two features make the method relevant to this thesis: (i) the straight-line ODE is by construction the trajectory any one-step sampler is trying to approximate, so the performance gap between 1-step and 10-step sampling is much smaller than for curved diffusion ODEs; and (ii) training is simulation-free and requires no teacher — FlowCast is an alternative to distillation rather than a distillation method. In Section 3.7 we adapt FlowCast to the StormCast residual pipeline and compare it directly to the distilled students on the same 2022 Taiwan validation year.

2.9 Why Distill?

Given a well-trained diffusion teacher, there are three broad ways to reduce the NFE budget at inference time.

1. **Higher-order and adaptive samplers.** DDIM [22], DPM-Solver [12], and the EDM Heun sampler itself reduce NFE by integrating the PF-ODE with better quadrature. These methods push NFE from thousands (ancestral sampling) down to 20–50. Below that the approximation error of the ODE integrator becomes visible.
2. **Distillation.** A student network is trained to match the teacher’s sampling output in fewer steps. Progressive Distillation and Consistency Distillation both fall in this

category and are the focus of this thesis.

3. **One-step generators.** Rectified flow [11] and shortcut models transform the probability path so that straight-line integration suffices. These methods are less mature for conditional forecasting and are not considered further.



2.10 Progressive Distillation

(author?) [19] introduced progressive distillation as a scheme to halve the number of sampling steps of a trained diffusion model. The core idea is to train a student to match, in one step, the result of two consecutive teacher steps. Specifically, fix a student-teacher pair at a target schedule with N steps. Sample a noised point x_t at noise level σ_t ; apply two teacher DDIM (or Heun) steps to obtain x_{t-2} at σ_{t-2} ; derive the implied denoised target $\tilde{x}$ by algebraically back-solving a one-step DDIM update from x_t to x_{t-2} ; and train the student denoiser to predict $\tilde{x}$ from (x_t, σ_t) . Once training converges, the student is promoted to be the new teacher, the schedule is halved again, and the procedure iterates. The original paper reports successful distillation down to as few as four and even a single sampling step on unconditional image benchmarks. Salimans and Ho also advocate a v -prediction parameterisation, which they find more numerically stable than ϵ -prediction in the low-NFE regime.

Progressive distillation makes no architectural changes and requires only that the student start from a copy of the teacher. It has the operational advantage of producing usable intermediate students at every schedule halving. The EDM variant of the same algorithm that we use is implemented in the denoised-prediction space (rather than v -space), which is more natural for the D_θ parameterisation of Section 2.6 and is the loss

formulation already present in our codebase.



2.11 Consistency Models and Consistency Distillation

(author?) [24] introduced a different family of few-step samplers based on a consistency function $f(x_\sigma, \sigma)$ with the defining property that, for any two noise levels $\sigma \geq \sigma' \geq \sigma_{\min}$ on the same PF-ODE trajectory,

$$f(x_\sigma, \sigma) = f(x_{\sigma'}, \sigma') = x_{\sigma_{\min}}. \quad (2.27)$$

The function is required to satisfy the boundary condition $f(x, \sigma_{\min}) = x$, which is enforced in practice by parameterising f through a skip connection — essentially an analogue of the EDM $c_{\text{skip}}, c_{\text{out}}$ scalings — so that the learned denoiser collapses to the identity at the smallest noise level.

Consistency distillation (CD). Given a teacher denoiser D_ψ , CD trains the student to satisfy the above identity along teacher-generated PF-ODE trajectories. Concretely, at each training step we sample $x_0 \sim p_{\text{data}}$ and two adjacent noise levels σ_n, σ_{n+1} from a discretisation $\{\sigma_0, \dots, \sigma_{N-1}\}$. We form the noised sample $x_{\sigma_{n+1}} = x_0 + \sigma_{n+1}\epsilon$, apply one teacher step Φ (a Heun update using D_ψ) to obtain an estimate of x_{σ_n} , and enforce

$$\text{dist}(f_\theta(x_{\sigma_{n+1}}, \sigma_{n+1}), f_{\theta^-}(\Phi(x_{\sigma_{n+1}}, \sigma_{n+1} \rightarrow \sigma_n), \sigma_n)) \rightarrow 0, \quad (2.28)$$

where θ^- is an EMA copy of the student weights and dist is a robust distance — typically a Pseudo-Huber loss $\text{dist}(a, b) = \sqrt{\|a - b\|^2 + c_h^2} - c_h$ with a small constant c_h .

Adaptive discretisation. (author?) [23] show that the trade-off between bias (from the teacher step) and variance (from the Monte-Carlo estimator) is best handled by growing the number of discretisation steps over training:

$$N(k) = \left\lceil \sqrt{\frac{k}{K} (N_{\text{total}}^2 - N_0^2) + N_0^2} \right\rceil, \quad (2.29)$$

where k is the current training iteration, K is the total, and N_0 and N_{total} are the starting and final discretisation sizes. The EMA decay is tied to $N(k)$ via $\mu_k = \mu_0^{N_0/N(k)}$, so that as the grid becomes finer the target network becomes tighter. Our CD implementation uses $N_0 = 2$ and $N_{\text{total}} = 150$.

Heun, not Euler, for Φ . The teacher operator Φ should be the same integrator that the teacher was trained to support. Because our EDM teacher uses the Heun sampler, Φ in our CD implementation is a single Heun step rather than the Euler step of the original consistency-model paper. Using Euler would introduce a systematic integration error proportional to $(\sigma_{n+1} - \sigma_n)^2$ that the student would learn to reproduce as a bias.

One-step and multi-step sampling. Once trained, the consistency model supports one-step sampling by drawing $x \sim \mathcal{N}(0, \sigma_{\text{max}}^2 I)$ and computing $f_\theta(x, \sigma_{\text{max}})$. Quality can be further improved by multi-step sampling, which re-noises the output at an intermediate σ and applies f_θ again, at the cost of one additional NFE per pass.



2.12 Adversarial Diffusion Distillation (Future Work)



A parallel line of work, Adversarial Diffusion Distillation (ADD) [20], combines a hinge GAN loss with a stop-gradient teacher-matching term to obtain high-quality one-to-four-step samplers for image synthesis. ADD requires a discriminator and is empirically sensitive to mode collapse, which is particularly dangerous for sparse heavy-tailed precipitation fields. An ADD scaffold exists in our codebase but is deferred to future work; we mention it here for completeness.

2.13 Prior Applications of Distillation to Weather

The literature on diffusion distillation applied specifically to weather forecasting is thin. Most existing work focuses on image synthesis and audio generation. Related efforts include distilled variants of DGMR for radar nowcasting, CorrDiff downscaling distillation experiments reported in NVIDIA PhysicsNeMo technical notes, and a small number of unpublished preprints on ensemble post-processing with few-step diffusion samplers. To the best of our knowledge, the present thesis is the first systematic study of progressive and consistency distillation applied to a regional convection-permitting diffusion forecaster, and the first to report precipitation-specific evaluation results under few-step sampling.

2.14 Evaluation of Precipitation Forecasts



Pixelwise RMSE alone is a poor measure of precipitation-forecast quality because it rewards smooth, low-amplitude predictions that miss extreme events. We adopt a suite of complementary metrics, each introduced briefly here and defined formally in Chapter 4.

Categorical skill scores. For a precipitation threshold τ , every pixel in the forecast and observation is classified as wet ($\geq \tau$) or dry, producing a 2×2 contingency table with hits H , misses M , false alarms F , and correct negatives. The Critical Success Index $CSI = H/(H + M + F)$ is the standard operational score. The frequency bias $(H + F)/(H + M)$ diagnoses over- or under-prediction.

Fractions Skill Score. (author?) [18] introduced FSS to handle the double-penalty problem: a forecast that is slightly displaced in space but correct in structure should not be penalised as heavily as one that is wrong everywhere. FSS computes the fraction of pixels exceeding τ in a neighbourhood of radius n around each point and compares forecast and observation fractions.

Spectral diagnostics. The radially averaged logarithmic power spectral density (log-PSD) of the forecast and observation fields quantifies sharpness. A deterministic model that produces blurry outputs will have a log-PSD that drops off too quickly at high wavenumbers; a generative model that matches the data distribution will track the observation spectrum.

Rollout error growth. For an autoregressive forecaster, single-step error understates the operational risk. We therefore report errors at rollout horizons of 1, 3, 6, and 12 hours, which captures the cumulative effect of distilled-student error on autoregressive predictions.

Probabilistic scores (optional). For ensemble forecasts we additionally report the continuous ranked probability score (CRPS) and rank histograms for qpepre.





Chapter 3 Method

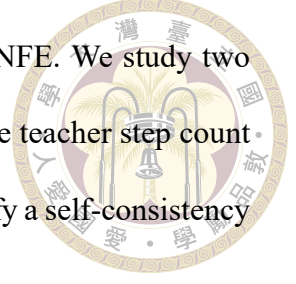
This chapter describes what was actually built. Section 3.1 states the distillation problem in precise terms. Section 3.2 describes the Taiwan RWR dataset and its normalisation. Section 3.3 documents the underlying data sources and the preprocessing pipeline that turns them into the training Zarr store. Section 3.4 recaps the frozen teacher. Sections 3.5 and 3.6 describe the progressive-distillation and consistency-distillation pipelines respectively. Section 3.7 describes FlowCast, a Conditional Flow Matching student that bypasses the teacher ODE entirely. Section 3.8 describes the dataset-specific loss modifications for the precipitation channel. Section 3.9 documents the training infrastructure.

3.1 Problem Setup

Let D_ψ denote the frozen EDM teacher diffusion network and F_ξ the frozen deterministic regression network, both provided as pretrained checkpoints. Given conditioning $c_t = \text{concat}(X_{t-1}, S_t, M_t, I)$, where $M_t = F_\xi(X_{t-1}, S_t, I)$, the teacher samples a residual $\hat{R}_t$ by integrating the PF-ODE from $\sigma_{\max}$ to $\sigma_{\min}$ using a Heun sampler with $N_{\text{teach}} \approx 13$ steps (so ≈ 25 NFE per hourly forecast step). The full prediction is $\hat{X}_t^{\text{teach}} = M_t + \hat{R}_t$.

The student D_θ shares the teacher’s architecture (an EDMPrecond SongUNet with the same conditioning interface) and is trained so that, given the same conditioning c_t ,

it produces a sample $\hat{X}_t^{\text{stud}} \approx \hat{X}_t^{\text{teach}}$ with at most $K \in \{1, 2, 4, 8\}$ NFE. We study two training procedures: progressive distillation, which directly halves the teacher step count in stages, and consistency distillation, which trains the student to satisfy a self-consistency property along teacher trajectories.



3.2 Dataset: Taiwan RWRF

The training data live in a Zarr store under the project directory (see Appendix A.5). The store has three top-level groups.

LowRes conditioning S_t . Shape $(T, 24, 224, 128)$, where T is the hourly time index and the 24 channels are mean sea-level pressure, two-metre temperature, ten-metre winds, and the specific humidity, temperature, and zonal and meridional winds at four pressure levels (1000 hPa, 850 hPa, 500 hPa and 250 hPa). These channels are low-resolution re-gridded ERA5 fields that serve as synoptic-scale boundary conditioning.

HighRes target X_t . Shape $(T, 4, 224, 128)$, with four channels: two-metre temperature (t2m), ten-metre zonal and meridional winds (u10, v10), and hourly accumulated precipitation (qpepre). The horizontal grid covers Taiwan from 21.6° to 25.6°N and 119.75° to 122.25°E at approximately 2 km resolution. The dataset splits into 21,216 training hours (2019-08 through 2021-12) and 8,760 validation hours (calendar year 2022).

Invariants I . Shape $(2, 224, 128)$, channels land-sea mask and orography. These are broadcast across time and concatenated into the conditioning bundle.

Normalisation. Per-channel means and standard deviations are computed on the training split and stored in the Zarr metadata. The HighRes statistics are

$$\mu_{\text{HR}} = (295.98, -1.58, -2.65, 0.182), \quad \sigma_{\text{HR}} = (5.61, 3.84, 5.66, 9.12), \quad (3.1)$$

which show that qpepre has a near-zero mean with standard deviation an order of magnitude larger than its mean — a direct fingerprint of its heavy-tailed, sparse distribution. Training and validation losses are computed on normalised fields; metrics in Chapter 4 are reported on denormalised fields with physical units.

3.3 Data Sources and Preprocessing Pipeline

Section 3.2 described the Zarr store consumed by the training loop. This section describes where the underlying fields come from and how the preprocessing pipeline in `data_preprocessing/` assembles them. The design follows a low-resolution guidance, high-resolution learning strategy: a coarse global reanalysis provides synoptic-scale conditioning, and a convection-permitting regional model plus a radar-derived precipitation product provides the fine-scale target.

3.3.1 Raw data sources

ERA5 (low-resolution input, S_t). The ECMWF ERA5 reanalysis [5] at approximately 25 km spatial resolution supplies the synoptic-scale environmental state. From ERA5 we take four surface variables (mean sea-level pressure `mslp`, two-metre temperature `t2m`, and ten-metre winds `u10`, `v10`) and five upper-air variables (specific humidity `q`, temperature `t`, zonal wind `u`, meridional wind `v`, and geopotential height `z`) at four pressure levels

{1000, 850, 500, 250} hPa, for a total of $4 + 5 \times 4 = 24$ channels. These are the 24 LowRes channels listed in Section 3.2.



RWRF (high-resolution target, part of X_t). The Radar-assimilated Weather Research and Forecasting (RWRF) numerical model provides convection-permitting (≈ 2 km) analyses over Taiwan. We use hourly RWRF output (wrfout_d01*_interp) for the three near-surface dynamical channels: t2m, u10, v10. RWRF contains the fine-scale dynamic and thermodynamic structures that the complex Taiwan terrain imprints on near-surface weather, which is exactly what a regional convective-scale forecaster must learn.

QPEPRE (high-resolution target, precipitation channel). Quantitative Precipitation Estimation (QPEPRE) is a radar-based hourly-rainfall product distributed as fixed-grid text files (qpepre_YYYYMMDDHHMM-YYYYMMDDHHMM_1_h.txt), one per hour. Unlike the RWRF fields, which are model outputs, QPEPRE is an observational product and therefore serves as a more faithful ground truth for surface rainfall than any model-forecast precipitation field. It occupies the fourth HighRes channel, qpepre.

Invariants I . Two static fields are extracted once from an RWRF file and broadcast across time: the land–sea mask lsm and orography orog. These let the network condition on terrain without re-reading static fields at every time step.

Table 3.1 summarises the three sources.

3.3.2 Preprocessing pipeline

The project ships two preprocessing stages under data_preprocessing/:

Table 3.1: Raw data sources used for training. Low-resolution ERA5 provides synoptic conditioning; RWRf and QPEPRE together supply the convection-permitting target.

	ERA5	RWRf	QPEPRE
Type	Reanalysis	Numerical model	Radar observation
Resolution	≈ 25 km	≈ 2 km	≈ 2 km
Role	LowRes input S_t	HighRes target X_t	HighRes target X_t
Channels used	24 (4 sfc + 5×4 levels)	3 (t2m, u10, v10)	1 (qpepre)
Native format	NetCDF	NetCDF (wrfout)	ASCII text

1. `nc_month_to_day/` — split monthly ERA5 NetCDF files into single-hour files.
2. `nc_to_zarr/` — harmonise ERA5, RWRf, and QPEPRE onto a common Taiwan grid and write a StormCast-style Zarr store.

Stage 1: monthly-to-hourly split. ERA5 is distributed as monthly NetCDF files, each containing an hourly time axis. The script `split_era5_monthly_to_daily.py` walks a root directory, opens every file matching a configured pattern, decodes the time axis with `netCDF4.num2date`, and writes one file per hour with a canonical `{variable}_YYYYMMDDTHH.nc` filename. The output uses uniform hours since 1970-01-01 00:00:00 time units so that downstream indexing does not have to handle per-file calendar differences. The split runs in parallel via a `multiprocessing.Pool` whose worker count is configurable (typically 20 processes on a workstation). Driver: `nc_month_to_day/split_era5_monthly_to_daily.py`; configuration: `nc_month_to_day/config.yaml`.

Stage 2: unified NetCDF-to-Zarr assembly. The main pipeline is `nc_to_zarr/nc_to_zarr_pipeline` driven by `config.yaml`. It is parameterised by four paths and a domain description:

`era5-path, wrf-path, qpepre-path, output-path`

`domain-size: [224, 128]`

`lon-bounds: [119.75, 122.25]`

lat-bounds: [21.6, 25.6]

train-ranges / valid-ranges



For each requested split (train, valid, or combined), the pipeline performs the following steps.

1. **Time-stamp generation.** The configured date ranges are expanded into a dense hourly list between the start and end of each range.
2. **File indexing.** Three in-memory indices are built — one per source — that map timestamps to absolute file paths. For ERA5 this is a nested {variable → {date-key → path}} dictionary built by a regex on filenames; for RWRF it is {folder-name → path}; for QPEPRE it is {time-key → path} derived from the qpepre_{start}-{end}_1_h.txt filename convention. When multiprocessing is enabled the indices are serialised to a single JSON file `.shared_index.json` in the output directory so that worker processes can mmap-read the index instead of receiving a pickled copy.
3. **Combined validity precomputation.** Before processing anything, the pipeline walks the timestamp list once and flags every hour for which any of the required ERA5 variables, RWRF file, or QPEPRE file is missing. Only timesteps that pass this joint check are processed, and the Zarr store carries an explicit per-timestamp valid mask so that downstream code can skip gaps without re-scanning the raw files.
4. **Per-timestep field extraction.**

- ERA5: open the NetCDF, select the target time index, select the correct pres-

sure level from the level dimension (for upper-air variables), crop to the latitude/longitude bounds, and resample to the configured 224×128 target grid.

- RWRF: read XLAT and XLONG to cache the native latitude/longitude, find the target Time index from the Times character variable, slice each requested variable to the domain, optionally apply per-variable transforms, and resample to the target grid.
- QPEPRE: load the text file, parse it into a {lat, lon, rain} scatter, grid it onto the native QPEPRE resolution, and resample to the 224×128 target grid. Because QPEPRE is already an observational product, we use it directly as the qpepre HighRes channel rather than the RWRF precipitation field.

5. **Grid harmonisation.** All three sources are resampled to a shared $H \times W = 224 \times 128$ grid covering lat $[21.6, 25.6]^\circ\text{N}$, lon $[119.75, 122.25]^\circ\text{E}$ at approximately 2 km spacing. This guarantees that the LowRes, HighRes, and invariant arrays share a pixel-aligned coordinate system, which is essential because the diffusion network conditions on LowRes and invariants by channel-wise concatenation.
6. **Streaming statistics.** As each training sample is filled into the data cube, a StreamingStats accumulator updates per-channel running means and standard deviations in single-pass Welford form. After the train split finishes, the stats are written to stats/means.npy and stats/stds.npy alongside the Zarr store. These are the normalisation statistics quoted in Equation 3.1.
7. **Zarr writing.** The assembled cube is written as an xarray.Dataset with dimensions (time, channel, y, x), chunked along time and space for efficient random access, and consolidated (zarr.consolidate_metadata) so that the train-

ing loader can open the store with a single metadata read. Invariants are written once, without a time dimension, as a separate Zarr store under `invariants/invariants.zarr`.



Output layout. A successful run produces the tree

`<output-path>/`

`LowRes/`

`stormcast_test_train.zarr      # (T_train, 24, 224, 128)`

`stormcast_test_valid.zarr      # (T_valid, 24, 224, 128)`

`stats/{means.npy, stds.npy, channels.txt}`

`HighRes/`

`stormcast_test_train.zarr      # (T_train, 4, 224, 128)`

`stormcast_test_valid.zarr      # (T_valid, 4, 224, 128)`

`stats/{means.npy, stds.npy, channels.txt}`

`invariants/`

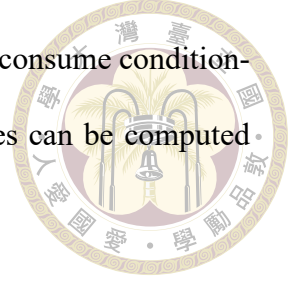
`invariants.zarr                  # (2, 224, 128)`

which matches the layout documented in `zarr_metadata.json` and consumed by `data_loader_rwrf_era`

For the experiment used in this thesis the pipeline produces $T_{\text{train}} = 21,216$ hourly samples (2019-08-01 through 2021-12-31) and $T_{\text{valid}} = 8,760$ hourly samples (calendar year 2022).

Why this matters for distillation. Three features of this pipeline are load-bearing for the work in this thesis:

- Pixel-aligned multi-source grids. The student and teacher both consume conditioning and target on the same 224×128 grid, so distillation losses can be computed pixel-wise without any on-the-fly re-gridding.
- Explicit valid mask. Roughly 10–15 % of hours fail the joint-validity check (usually a missing QPEPRE file). Both loaders and metrics respect the mask, so downstream scores are never contaminated by NaN-filled cells.
- Frozen training-split statistics. Because the Zarr store commits to a single pair of $(\mu_{\text{HR}}, \sigma_{\text{HR}})$ at write time, teacher and student see identical normalised inputs — a prerequisite for the preconditioning-matching discussion in Section 3.4.



3.3.3 Target Distributions: Sparsity, Gaussianity, and the Diffusion Prior

Diffusion models place a Gaussian prior on their target: the forward process is $x_\sigma = x_0 + \sigma \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, I)$, and the denoiser is trained under the implicit assumption that the marginals of x_0 are reasonably well-behaved relative to that noise scale. How closely each of the four HighRes channels actually conforms to this assumption directly shapes how the distilled student must be regularised. Figure 3.1 shows per-channel histograms (with a fitted Gaussian overlay) and Gaussian QQ plots computed over the full training split ($n = 6.08 \times 10^8$ pixels per channel). The streaming statistics are summarised in Table 3.2.

The three near-surface state channels are approximately Gaussian. u10 is the closest: almost zero skewness, mild leptokurtosis, and a QQ plot that tracks the diagonal out to roughly $\pm 4\sigma$ before fanning into the heavy tails associated with gust events. v10 is almost

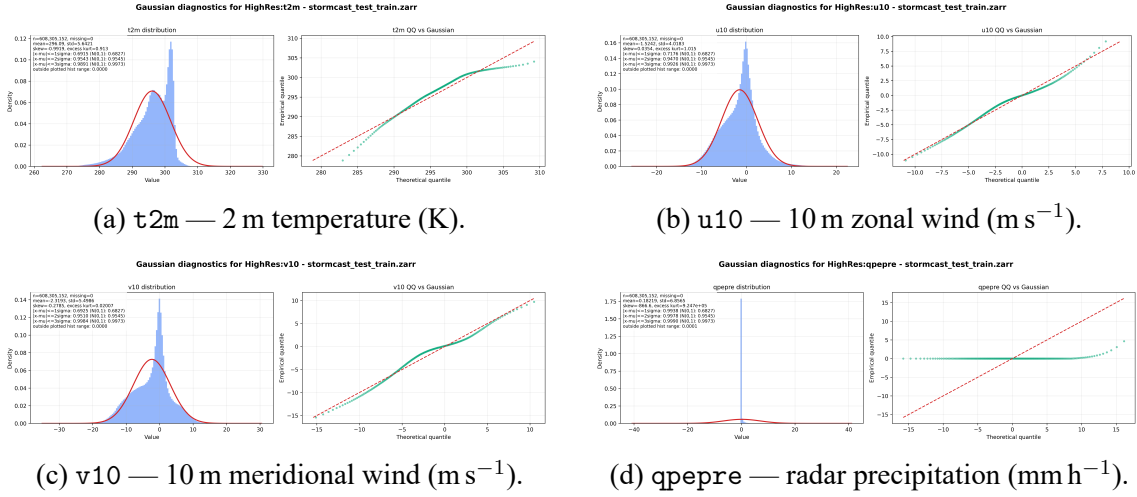
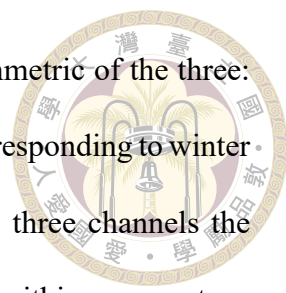


Figure 3.1: Per-channel distribution and Gaussian QQ plots for the four HighRes target channels over the 2.5-year training split. Left subpanels: empirical density (red) with a Gaussian fit of matched (μ , σ) (blue). Right subpanels: empirical quantiles against theoretical Gaussian quantiles; the red dashed line is $y = x$ for a perfect Gaussian. Generated by `data_preprocessing/inspect_tools/plot_high_res_vars.py`.

Table 3.2: Summary statistics of the four HighRes target channels over the training split. A standard Gaussian has skewness = 0, excess kurtosis = 0, and concentrates 68.3/95.5/99.7% of its mass within $1/2/3\sigma$.

Channel	Mean	Std	Skewness	Excess kurt.	% in 1σ	% in 3σ
$t2m$	296.09	5.64	-0.99	0.91	69.2	98.9
$u10$	-1.52	4.02	0.04	1.01	71.8	99.3
$v10$	-2.32	5.50	-0.28	0.02	69.3	99.8
$qpepre$	0.18	6.86	-866.6	9.25×10^5	99.4	99.9



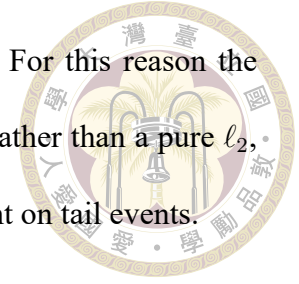
symmetric with essentially zero excess kurtosis. t_{2m} is the least symmetric of the three: left-skewed (skew = -0.99) with a secondary mode around 280 K corresponding to winter cold surges over northern Taiwan and the surrounding sea. For all three channels the empirical $1\sigma/2\sigma/3\sigma$ coverage agrees with the Gaussian reference to within a percentage point, which is exactly the regime for which the EDM noise schedule with $\sigma_{\text{data}} = 0.5$ was designed: the teacher denoiser sees each channel as a mild deformation of a unit-variance Gaussian after normalisation, and the standard ℓ_2 denoising loss is a reasonable fit.

qppepre is a different object entirely. Its marginal is dominated by an atom at zero — the overwhelming majority of pixel-hours are dry — with a long, very sparse tail of heavy rainfall events. The raw statistics make this quantitative: the coverage inside $\pm 1\sigma$ is 99.4 % rather than the Gaussian 68.3 %, which is the signature of a near-delta distribution whose standard deviation is inflated entirely by the tail. The empirical moments reported in Table 3.2 (skewness ≈ -867 , excess kurtosis $\approx 10^6$) are formally enormous because the raw qppepre field uses a -9999 sentinel for missing observations that is only filtered at the next pipeline stage; the physical tail above zero is still extreme (excursions beyond 100 mm h^{-1} occur during typhoon landfalls), but the numbers headline the practical message: the qppepre histogram looks nothing like a Gaussian, and the QQ plot sits on a flat shelf near zero that peels almost vertically at the upper end.

This mismatch has three consequences for diffusion-based distillation that recur throughout the rest of this thesis:

1. Score matching on raw qppepre is ill-posed at small σ . When the target distribution is almost a point mass at zero, the score $\nabla_x \log p_\sigma(x)$ diverges as $\sigma \rightarrow \sigma_{\min}$. The teacher partially absorbs this by learning the residual relative to the regression mean

M_t , but the residual inherits the same atom-plus-tail shape. For this reason the consistency loss in Section 3.6 uses a Pseudo-Huber penalty rather than a pure ℓ_2 , trading a small amount of bias near zero for a bounded gradient on tail events.



2. Pixel-wise losses undersell the tail. Because dry pixels vastly outnumber rainy ones, an ℓ_2 loss converges to a “slightly wet everywhere” solution that scores well on RMSE but destroys the convective structure we actually care about. This is the motivation for the radial log-PSD spectral regulariser applied specifically to qppepre in both the consistency and ADD losses, and for the precipitation-specific metrics (CSI, FSS, frequency-of-exceedance at fixed rainfall thresholds) reported in Chapter 4.
3. The channel budget matters. Because only one of the four output channels is pathological, per-channel loss weights β_k are a natural knob: the three near-Gaussian fields can be trained with a standard denoising objective while qppepre receives heavier regularisation and, if needed, an asinh or $\log(1 + x)$ transform to compress its dynamic range before the noise is added. Having only four target channels (versus the 99 of global StormCast) makes such per-channel bespoke treatment computationally trivial.

In short, Figure 3.1 is the quantitative justification for every “qppepre is the hard channel” design decision that follows: three channels live comfortably inside the Gaussian prior that diffusion assumes, and one does not.

3.4 Teacher Architecture Recap



The teacher is the two-stage StormCast model described in Section 2.3, with the EDM hyperparameters of Section 2.6.

- **Regression** F_ξ . A StormCastUNet taking (X_{t-1}, S_t, I) as input and predicting M_t .
The checkpoint StormCastUNet.0.14000.md1us is used without modification or further training.
- **Diffusion** D_ψ . An EDMPrecond SongUNet trained to denoise the residual $R_t = X_t - M_t$ conditioned on c_t . The checkpoint EDMPrecond.0.70000.md1us is used as the distillation teacher. EDM hyperparameters: $\sigma_{\min} = 0.002$, $\sigma_{\max} = 80$, $\sigma_{\text{data}} = 0.5$, $\rho = 7$.

Both checkpoints are loaded through the helper `get_preconditioned_architecture` in `stormcast/utils/nn.py`, which instantiates the network and the EDM wrapper. The conditioning bundle is assembled by `build_network_condition_and_target`, which handles the concatenation and the one-hot time-offset encoding used in the rollout.

Exact hyperparameter matching. An early implementation mistake was to initialise the student with EDM hyperparameters that differed from the teacher by small amounts (for example, using $\sigma_{\text{data}} = 1.0$ instead of 0.5). This silently broke distillation: the student’s preconditioning scalings no longer aligned with the teacher’s, so the loss targets were in the wrong numerical range. We therefore copy teacher hyperparameters verbatim into student configs and verify them at load time.

3.5 Method A: Progressive Distillation



Our progressive-distillation pipeline follows Salimans and Ho’s Algorithm 2 but operates in the EDM denoised-prediction space of D_θ rather than in the v -prediction space of the original paper. This matches the teacher’s parameterisation and removes the need to convert between spaces at every step.

Phase structure. Training proceeds through a sequence of phases. Phase p starts with a student–teacher pair sharing a schedule of N_p steps; at the end of the phase, the schedule is halved to $N_{p+1} = N_p/2$ and the student is promoted to be the new teacher. Our default phase schedule is $N_0 = 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$, producing five phases. Each phase runs for a configurable number of training steps (the default is 6×10^4).

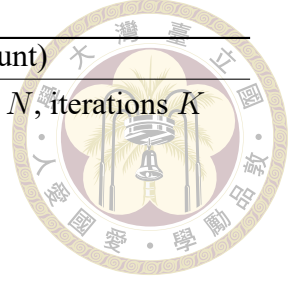
Per-step loss. Within a phase, each training step samples a mini-batch of conditioning bundles c_t , samples a noise level σ_t from the EDM log-normal, forms the noised target $x_{\sigma_t} = R_t + \sigma_t \epsilon$, and then applies two teacher Heun steps to obtain $x_{\sigma_{t-2}}$. The implied one-step target $\tilde{x}$ is recovered by inverting the DDIM update

$$\tilde{x} = \frac{x_{\sigma_t} - (\sigma_{t-2}/\sigma_t) x_{\sigma_{t-2}}}{1 - \sigma_{t-2}/\sigma_t}. \quad (3.2)$$

The student loss is

$$\mathcal{L}_{\text{PD}} = \mathbb{E} \left[w(\sigma_t) \| D_\theta(x_{\sigma_t}, \sigma_t; c_t) - \tilde{x} \|_{2,\beta}^2 \right], \quad (3.3)$$

where $w(\sigma_t)$ is the EDM loss weighting inherited from the teacher and $\| \cdot \|_{2,\beta}$ is the channel-weighted L2 norm defined in Section 3.8.



Algorithm 1 Progressive-distillation phase (single halving of step count)

Require: teacher D_ψ , student D_θ initialised from teacher, step count N , iterations K

- 1: **for** $k = 1, \dots, K$ **do**
 - 2: sample mini-batch $\{(X_{t-1}, S_t, X_t)\}$; form M_t, R_t, c_t
 - 3: sample noise level $\sigma_t \sim \text{LogNormal}$, draw $\epsilon \sim \mathcal{N}(0, I)$
 - 4: $x_{\sigma_t} \leftarrow R_t + \sigma_t \epsilon$
 - 5: $x_{\sigma_{t-1}} \leftarrow \text{Heun}(D_\psi, x_{\sigma_t}, \sigma_t \rightarrow \sigma_{t-1}; c_t)$
 - 6: $x_{\sigma_{t-2}} \leftarrow \text{Heun}(D_\psi, x_{\sigma_{t-1}}, \sigma_{t-1} \rightarrow \sigma_{t-2}; c_t)$
 - 7: $\tilde{x} \leftarrow (x_{\sigma_t} - (\sigma_{t-2}/\sigma_t) x_{\sigma_{t-2}}) / (1 - \sigma_{t-2}/\sigma_t)$
 - 8: $\mathcal{L} \leftarrow w(\sigma_t) \|D_\theta(x_{\sigma_t}, \sigma_t; c_t) - \tilde{x}\|_{2,\beta}^2$
 - 9: update student parameters with Adam on $\mathcal{L}$
 - 10: **end for**
 - 11: promote student to teacher, set $N \leftarrow N/2$
-

Phase transitions. At the end of each phase we (i) copy the student weights into a fresh teacher instance, (ii) halve N , and (iii) call `set_num_steps(N)` on the loss module — a subtle but important point, because without this call the loss still computes expectations against the previous schedule and training silently fails.

Algorithmic summary. Algorithm 1 gives the per-phase training loop in pseudocode form.

Implementation pointers. The entry point is `stormcast/train_progressive.py`; the per-phase loop lives in `stormcast/utils/trainer_progressive.py` as `progressive_distillation`; the loss is at `physicsnemo/experimental/metrics/diffusion/progressive_distillation_loss.py`; configuration is in `stormcast/config/progressive.yaml`. Inference uses `progressive_distilled_fm` from `stormcast/utils/nn.py`, which replaces the Heun sampler with the student’s learned few-step schedule.

3.6 Method B: Consistency Distillation



Consistency distillation trains D_θ to be a consistency function along teacher-generated PF-ODE trajectories, as described in Section 2.11. Our implementation wraps the student in a `ConsistencyPrecond` module that enforces the boundary condition $f(x, \sigma_{\min}) = x$ by the same skip-connection trick as EDM; at $\sigma = \sigma_{\min}$ the skip scaling is 1 and the output scaling is 0, so the wrapper is the identity.

Per-step loss. At training iteration k we read the current discretisation size $N(k)$ (see below) and sample an index $n \in \{0, \dots, N(k) - 2\}$ uniformly. We then sample $x_0 \sim p_{\text{data}}$, form $x_{\sigma_{n+1}} = R_t + \sigma_{n+1}\epsilon$, and apply one teacher Heun step Φ to produce $x_{\sigma_n} = \Phi(x_{\sigma_{n+1}}, \sigma_{n+1} \rightarrow \sigma_n; c_t, D_\psi)$. The loss is the channel-weighted Pseudo-Huber distance

$$\mathcal{L}_{\text{CD}} = \mathbb{E} \left[\rho_{c_h, \beta} \left(f_\theta(x_{\sigma_{n+1}}, \sigma_{n+1}; c_t), f_{\theta^-}(x_{\sigma_n}, \sigma_n; c_t) \right) \right] + \lambda_{\text{spec}} \mathcal{L}_{\text{spec}}, \quad (3.4)$$

where $\rho_{c_h, \beta}(a, b) = \sum_k \beta_k (\sqrt{\|a_k - b_k\|^2 + c_h^2} - c_h)$ is the per-channel-weighted Pseudo-Huber distance, θ^- denotes the EMA copy of the student, and $\mathcal{L}_{\text{spec}}$ is the radial-PSD regulariser defined in Section 3.8. We use $c_h = 5.4 \times 10^{-4}$, as recommended by the improved-consistency-training paper.

Adaptive discretisation and EMA decay. We use the sqrt schedule

$$N(k) = \left\lceil \sqrt{\frac{k}{K} (N_{\text{total}}^2 - N_0^2) + N_0^2} \right\rceil, \quad \mu_k = \mu_0^{N_0/N(k)}, \quad (3.5)$$

with $N_0 = 2$, $N_{\text{total}} = 150$, $\mu_0 = 0.95$, and K equal to the total training iteration count.

Both quantities are implemented in `stormcast/utils/ema.py` as `ema_decay_schedule`

and ExponentialMovingAverage.



Heun teacher operator. Φ is a single Heun step using the frozen teacher, as argued in Section 2.11. This matches the integrator the teacher was trained to emulate and removes an Euler-step bias term the student would otherwise absorb.

Inference. One-step sampling draws $x \sim \mathcal{N}(0, \sigma_{\max}^2 I)$ and returns $f_{\theta}(x, \sigma_{\max}; c_t)$. Two- and four-step sampling alternates f_{θ} with additive re-noising at a chosen intermediate σ . Critically, the inference weights are the EMA shadow parameters, not the online student: loading `checkpoint_*.pt` directly yields worse samples than loading `ema_state.pt`, and our evaluation scripts load the EMA file by default.

Implementation pointers. Entry point `stormcast/train_consistency.py`; training loop `stormcast/utils/trainer_consistency.py`; loss `physicsnemo/experimental/metrics/diffusion/consistency_loss.py` (`ConsistencyDistillationLoss`); configs `stormcast/config/consistency.yaml` and `stormcast/config/training/consistency.yaml`.

3.7 Method C: FlowCast (Conditional Flow Matching)

FlowCast replaces the teacher PF-ODE with a simulation-free straight-line flow: a single network is trained from scratch to regress the target vector field $u_t = x_1 - x_0$ of Independent CFM (Section 2.8), and sampling is a handful of Euler steps. Unlike PD and CD, FlowCast is not a distillation method — it does not use the EDM teacher at all — but it is included in this thesis as a direct one-training-run alternative to few-step distilled sampling that exposes what is attributable to the teacher versus what a straight-line prior already de-

livers. The same StormCast residual decomposition is preserved: the deterministic regression mean M_t is produced by the frozen F_ξ , and FlowCast learns only the residual $R_t = X_t - M_t$ conditioned on a bundle c_t assembled by `build_network_condition` and target in `stormcast/utils/nn.py`.

Conditioning bundle. The conditioning channels FlowCast sees differ slightly from the EDM teacher's. The training config `stormcast/config/model/flowcast.yaml` sets

```
regression_conditions: ["state", "background", "invariant"]
diffusion_conditions: ["state", "regression", "invariant"]
```

so the frozen regression network F_ξ is fed the full (X_{t-1}, S_t, I) as before, but the flow-matching network itself is conditioned only on

$$c_t = \text{concat}(X_{t-1}, M_t, I), \quad (3.6)$$

with the low-resolution background S_t deliberately omitted from the direct conditioning path. The synoptic-scale signal in S_t therefore enters FlowCast only through the regression mean $M_t = F_\xi(X_{t-1}, S_t, I)$. This trims the conditioning-input channel count by 24 at zero cost to the flow matching objective, because M_t already summarises what the residual needs to know about S_t ; keeping "regression" in the diffusion-conditions list is what keeps the FlowCast residual anchored to exactly the same M_t the EDM teacher and distilled students are anchored to, which is what makes the cross-method comparison of Chapter 4 fair.

Standardised-residual pipeline. FlowCast operates on the residual in a standardised space. The loss (`stormcast/utils/flowcast_loss.py`) divides the raw residual by $\sigma_{\text{data}} = 0.5$ before building the flow trajectory, and the sampler (`stormcast/utils/nn.py:flowcast_model_forward`) multiplies the integrated state by σ_{data} on output. Keeping σ_{data} matched to the EDM teacher means the regression-mean anchoring statistics and the per-channel loss scaling are identical across all three methods. Concretely, with $x_1 = R_t/\sigma_{\text{data}}$ and $x_0 \sim \mathcal{N}(0, I)$, the I-CFM interpolated state is

$$x_t = (1 - t) x_0 + t x_1 + \sigma_{\text{path}} \eta, \quad t \sim \mathcal{U}(t_\epsilon, 1 - t_\epsilon), \quad \eta \sim \mathcal{N}(0, I), \quad (3.7)$$

with $\sigma_{\text{path}} = 0.01$ from (author?) [26] and $t_\epsilon = 10^{-5}$ clamping the endpoints. The small $\sigma_{\text{path}}\eta$ perturbation thickens the otherwise-Dirac conditional path $p_t(x_t \mid x_0, x_1)$ and is what separates I-CFM from the original deterministic Optimal-Transport CFM of (author?) [10].

Velocity predictor. The student architecture is the FlowCastPrecond module at `stormcast/`
`utils/flowcast_precond.py`, instantiated through the factory call `get_preconditioned_architecture(...)` at `stormcast/utils/nn.py`. The factory wraps a SongUNet with `channel_mult=[1, 2, 2, 2, 2]`, identical to the EDM teacher’s backbone. Unlike EDMPrecond there are no $c_{\text{skip}}/c_{\text{in}}/c_{\text{out}}$ scalings — the network output is the velocity estimate:

$$v_\theta(x_t, t, c_t) = \text{SongUNet}(\text{concat}[x_t, c_t], t \cdot \tau), \quad (3.8)$$

where $\tau = 1000$ (the `time_scale` field in `stormcast/config/model/flowcast.yaml`) rescales the flow time $t \in [0, 1]$ before it enters the SongUNet’s positional-embedding layer. The SongUNet positional embedding was designed for diffusion-timestep-valued

Algorithm 2 FlowCast training step (I-CFM on the standardised residual)

Require: frozen F_ξ , FlowCast student v_θ , schedule $(\sigma_{\text{data}}, \sigma_{\text{path}}, t_\varepsilon)$, channel weights β , spectral weight λ_{spec}

- 1: sample mini-batch $\{(X_{t-1}, S_t, X_t)\}$; form $M_t \leftarrow F_\xi(X_{t-1}, S_t, I)$, $R_t \leftarrow X_t - M_t$
- 2: build conditioning $c_t \leftarrow \text{concat}(X_{t-1}, M_t, I)$ $\triangleright$ note: S_t reaches v_θ only via M_t
- 3: standardise: $x_1 \leftarrow R_t / \sigma_{\text{data}}$
- 4: sample $x_0 \sim \mathcal{N}(0, I)$, $t \sim \mathcal{U}(t_\varepsilon, 1 - t_\varepsilon)$, $\eta \sim \mathcal{N}(0, I)$
- 5: build interpolant: $x_t \leftarrow (1 - t) x_0 + t x_1 + \sigma_{\text{path}} \eta$
- 6: target field: $u_t \leftarrow x_1 - x_0$
- 7: predict: $\hat{v} \leftarrow v_\theta(x_t, t, c_t)$
- 8: pointwise: $\mathcal{L}_{\text{pw}} \leftarrow \|\hat{v} - u_t\|_{2,\beta}^2$
- 9: implied clean: $\hat{x}_1 \leftarrow x_t + (1 - t) \hat{v}$
- 10: spectral: $\mathcal{L}_{\text{spec}} \leftarrow \frac{1}{|\mathcal{K}|} \sum_{\kappa} |\log P_{\hat{x}_1^{\text{qpepre}}}(\kappa) - \log P_{x_1^{\text{qpepre}}}(\kappa)|$
- 11: $\mathcal{L} \leftarrow \mathcal{L}_{\text{pw}} + \lambda_{\text{spec}} \mathcal{L}_{\text{spec}}$
- 12: AdamW step on θ ; EMA update $\theta^- \leftarrow 0.999 \theta^- + 0.001 \theta$

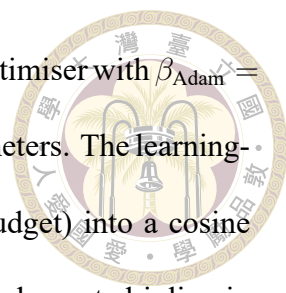
inputs with dynamic range of order 10^2 – 10^3 ; feeding raw $t \in [0, 1]$ would collapse the embedding to a nearly-constant signal. $\tau = 1000$ restores enough dynamic range for the embedding to carry useful time information without changing the SongUNet itself, so no architectural changes from the teacher are needed.

Loss. The training loss is the standardised-space I-CFM regression

$$\mathcal{L}_{\text{FC}} = \mathbb{E}[\|v_\theta(x_t, t, c_t) - (x_1 - x_0)\|_{2,\beta}^2] + \lambda_{\text{spec}} \mathcal{L}_{\text{spec}}(\hat{x}_1, x_1), \quad (3.9)$$

where $\|\cdot\|_{2,\beta}$ is the per-channel-weighted L2 norm from Section 3.8 with $\beta = (1, 1, 1, 2)$ (the `channel_weights` field), $\hat{x}_1 = x_t + (1 - t) v_\theta$ is the flow’s implied clean-residual estimate at the current time, and $\mathcal{L}_{\text{spec}}$ is the radial log-PSD regulariser of Section 3.8 with `spectral_channels` = {`qpepre`} and $\lambda_{\text{spec}} = 0.1$. Operating the spectral term on the implied-clean field $\hat{x}_1$ rather than on the raw velocity keeps the spectrum interpretable.

Algorithm 2 summarises the per-step update.



Optimisation. Following the FlowCast paper we use the AdamW optimiser with $\beta_{\text{Adam}} = (0.9, 0.999)$, learning rate 5×10^{-4} , and weight decay 10^{-4} on all parameters. The learning-rate schedule is a 4,000-step linear warmup ($\sim 1\%$ of the training budget) into a cosine decay to $0.01 \times 5 \times 10^{-4} = 5 \times 10^{-6}$ over the remaining steps, implemented inline in `trainer_flowcast.py`. Gradients are globally clipped at 1.0, and per-parameter NaN gradients are sanitised with `torch.nan_to_num` (a safety net borrowed from the EDM trainer). Training runs for a total of 400,000 optimiser steps at a global batch size of 64, split across the available GPUs through PyTorch’s `DistributedDataParallel`; the per-GPU micro-batch is `batch_size_per_gpu = "auto"`, which the trainer resolves to $64/W$ for W workers.

EMA target network. A second `FlowCastPrecond` instance is kept as an exponential-moving-average shadow of the online student, with a fixed decay of 0.999. The shadow is updated after every optimiser step by `ExponentialMovingAverage.update` and copied into the shadow network by `apply_shadow` (`stormcast/utils/ema.py`). Unlike CD, where the decay μ_k is coupled to the adaptive discretisation $N(k)$, FlowCast uses a single decay throughout training because the training target (the vector field) is fixed and does not evolve with a shrinking time discretisation. The shadow weights, not the online student, are used at validation and inference time, and are saved to `ema_state.pt` alongside each regular checkpoint. Loading the raw `checkpoint_*.pt` instead of `ema_state.pt` yields visibly noisier samples.

Sampling. Generation integrates the learned ODE $\dot{x}_t = v_\theta(x_t, t, c_t)$ from $t = 0$ to $t = 1$ with a fixed-step solver. The implementation `flowcast_model_forward` in `stormcast/`

utils/nn.py exposes two choices. The default is explicit Euler with K steps:

$$x_{t+\Delta t} = x_t + \Delta t v_\theta(x_t, t, c_t), \quad \Delta t = 1/K, \quad x_{t=0} \sim \mathcal{N}(0, I), \quad (3.10)$$

which costs K NFEs. The alternative `solver="midpoint"` is a single-stage Runge–Kutta second-order scheme:

$$v_{1/2} = v_\theta(x_t, t, c_t), \quad \tilde{x} = x_t + \frac{1}{2}\Delta t v_{1/2}, \quad x_{t+\Delta t} = x_t + \Delta t v_\theta(\tilde{x}, t + \frac{1}{2}\Delta t, c_t), \quad (3.11)$$

which costs $2K$ NFEs but has $O(\Delta t^2)$ local truncation error. The sampler finishes by de-standardising: $\hat{R}_t = \sigma_{\text{data}} x_{t=1}$ and $\hat{X}_t = M_t + \hat{R}_t$. Validation inside the training loop uses $K = \text{valid_num_steps} = 10$ Euler steps, while the main experiments sweep $K \in \{1, 2, 4, 8, 10\}$ against the distilled students at matched NFE counts.

Why include FlowCast as a baseline. Three properties of I-CFM make FlowCast the right control experiment for the distillation methods of Sections 3.5 and 3.6. First, it separates the “few-step sampling” question from the “distill from a teacher” question: any quality gap between FlowCast and the distilled students at matched NFE isolates the benefit of the teacher’s signal rather than the benefit of collapsing the sampling trajectory. Second, the straight-line probability path is by construction what a one-step deterministic generator is trying to approximate, so the FlowCast $K = 1$ number is a strong reference point for the best that a single-step ODE sampler trained on the same data can do. Third, and specific to precipitation, (author?) [17] report that I-CFM is noticeably more stable than score matching on sparse heavy-tailed fields — a claim directly relevant to the qpepre channel.

Implementation pointers. Entry point `stormcast/train_flowcast.py`; training loop `stormcast/utils/trainer_flowcast.py (flowcast_training_loop)`; loss `stormcast/utils/flowcast_loss.py (FlowCastLoss)`; preconditioner `stormcast/utils/flowcast_precond.py (FlowCastPrecond)`; EMA helper `stormcast/utils/ema.py (ExponentialMovingAverage)`; configs `stormcast/config/flowcast.yaml`, `stormcast/config/model/flowcast.yaml`, `stormcast/config/training/flowcast.yaml`; sampler `flowcast_model_forward` in `stormcast/utils/nn.py`. The numerical hyperparameters are tabulated in Appendix 5.4.

3.8 Loss Design for the Precipitation Channel

The four-channel RWRF target makes per-channel loss engineering cheap, and we use this budget to attack the hardest channel, `qppepre`, from two directions.

Channel-weighted Pseudo-Huber. Let $\beta = (\beta_{t2m}, \beta_{u10}, \beta_{v10}, \beta_{qppepre})$ be a vector of non-negative per-channel weights. The weighted L2 and Pseudo-Huber norms are

$$\|a\|_{2,\beta}^2 = \sum_{k=1}^4 \beta_k \|a_k\|^2, \quad \rho_{c_h,\beta}(a, b) = \sum_{k=1}^4 \beta_k (\sqrt{\|a_k - b_k\|^2 + c_h^2} - c_h), \quad (3.12)$$

where the sum runs over the four output channels and the norm inside each term is the ordinary spatial L2. The default configuration uses $\beta = (1, 1, 1, w_{pr})$ with $w_{pr} \in \{2, 4\}$ to compensate for the precipitation channel's smaller effective signal under the Pseudo-Huber loss. These weights are exposed as `channel_weights` in the consistency-loss config.

Radial log-PSD regulariser. Because RMSE-like losses reward smooth, low-amplitude precipitation outputs, we add a spectral-matching regulariser that penalises the L1 distance between the radial log-PSD of the predicted and target qpepre fields:

$$\mathcal{L}_{\text{spec}} = \frac{1}{|\mathcal{K}|} \sum_{\kappa \in \mathcal{K}} |\log P_{\text{pred}}(\kappa) - \log P_{\text{target}}(\kappa)|, \quad (3.13)$$

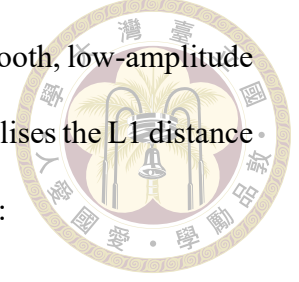
where $P(\kappa)$ is the radially averaged 2D power spectral density at wavenumber κ and $\mathcal{K}$ is the set of non-zero radial wavenumbers on the 224×128 grid. The regulariser is active only on channels listed in `spectral_channels` and weighted by `spectral_weight =  $\lambda_{\text{spec}}$` . By default we apply it to qpepre only.

Monitoring for mode collapse. Even with the above terms, sparse heavy-tailed precipitation is an invitation to mode collapse: the model can minimise both MSE and Pseudo-Huber by producing a slightly-wet constant field. During training we therefore log the wet-pixel fraction at thresholds $\{0.1, 1.0, 5.0\} \text{ mm h}^{-1}$ and the 95th and 99th percentiles of the output distribution, and compare them to the same quantities on the training batch. A large divergence is a collapse warning.

3.9 Training Infrastructure

All runs use PyTorch with `torchrun --standalone --nnodes=1 --nproc_per_node=N` for multi-GPU training. Configuration is managed with Hydra; experiment overrides are passed on the command line as key=value pairs. Checkpoints are written every M iterations (default $M = 1000$) under

`StormCast_{method}/{experiment}/0/checkpoints_{method}/`



checkpoint_*.pt



together with CSV loss logs and, every V iterations, validation heatmap and power-spectrum images for the four output channels. Auto-resume is implemented by `_detect_resume_state` in the progressive trainer and by a `checkpoint_latest.pt` convention in the consistency trainer. A run can therefore be restarted with the same command line and will pick up from the last written checkpoint.





Chapter 4 Experiments and Results

This chapter presents the empirical evaluation of the two distillation methods described in Chapter 3. Section 4.1 defines the experimental protocol. Section 4.2 defines the evaluation metrics in precise terms. Section 4.3 reports progressive-distillation results; Section 4.4 reports consistency-distillation results; Section 4.5 compares the two methods at matched NFE budgets. Section 4.6 presents qualitative case studies over selected 2022 Taiwan weather events. Section 4.7 catalogues failure modes, and Section 4.8 concludes with a discussion of when to prefer each method.

*Note: this chapter is structured as the final results chapter but populated with provisional numbers. Tables and figures flagged with **[TBD]** are to be filled in as the final eval runs complete.*

4.1 Experimental Protocol

Validation split. All reported metrics are computed on the 2022 full-year RWRF validation zarr (8,760 hourly samples). This contrasts with the shipped configuration in `stormcast/config/inference/stormcast.yaml`, whose `valid_dates` entry points at a different period; we override it explicitly in all runs.

NFE budgets. We evaluate the teacher at its native schedule (≈ 25 NFE) and the distilled students at $K \in \{1, 2, 4, 8\}$. For progressive distillation these correspond to the intermediate checkpoints produced at phase transitions. For consistency distillation, $K = 1$ is one-step sampling and $K \in \{2, 4, 8\}$ are multi-step samplers with re-noising.

Seeds and ensembling. Single-forecast metrics (Sections 4.3–4.5) use a fixed seed per forecast hour for determinism. Ensemble metrics (Section 4.8, optional) use 10 members per forecast hour with independent noise seeds.

Rollouts. All reported rollouts start from the validation initial condition at 0 UTC every fourth day and integrate forward autoregressively for 12 hours. The regression network is the same frozen checkpoint for teacher and students, so the only difference across methods is the diffusion network.

Hardware. Training: multi-GPU single-node, [TBD] NVIDIA A100 80 GB. Evaluation: single GPU. Exact wall-clock times are reported in Appendix A.5.

4.2 Evaluation Metrics

All metrics are computed on denormalised fields with physical units.

RMSE and MAE. For channel k ,

$$\text{RMSE}_k = \sqrt{\frac{1}{|\mathcal{T}|HW} \sum_{t \in \mathcal{T}} \sum_{i,j} (\hat{X}_{t,i,j}^{(k)} - X_{t,i,j}^{(k)})^2}, \quad (4.1)$$

with MAE defined analogously.

Categorical precipitation scores. For threshold τ (mm h^{-1}), classify each pixel as wet ($\geq \tau$) or dry. With hit H , miss M , false alarm F :

$$\text{CSI}(\tau) = \frac{H}{H + M + F}, \quad \text{FB}(\tau) = \frac{H + F}{H + M}. \quad (4.2)$$



We report $\tau \in \{0.1, 1.0, 5.0, 10.0, 20.0\} \text{ mm h}^{-1}$.

Fractions Skill Score. With neighbourhood radius n ,

$$\text{FSS}(\tau, n) = 1 - \frac{\sum_{i,j} (p_{i,j}^{\text{fc}} - p_{i,j}^{\text{ob}})^2}{\sum_{i,j} (p_{i,j}^{\text{fc}})^2 + \sum_{i,j} (p_{i,j}^{\text{ob}})^2}, \quad (4.3)$$

where $p_{i,j}$ is the fraction of τ -exceeding pixels in an $n \times n$ box centred at (i, j) . We report $n \in \{5, 11, 21\}$ grid points.

Radial log-PSD. We compute the 2D FFT of each qpepre field, square its magnitude, and radially average to obtain $P(\kappa)$, then compare $\log P_{\text{pred}}(\kappa)$ and $\log P_{\text{target}}(\kappa)$ on the same plot.

Rollout metrics. Rollout RMSE and CSI are computed as above but as a function of lead time $\ell \in \{1, 3, 6, 12\} \text{ h}$.

4.3 Progressive-Distillation Results

Training dynamics. Figure [TBD] shows per-phase training loss curves for PD with the $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ schedule. Each phase converges within [TBD] iterations; promoting the student to teacher at each boundary introduces a characteristic loss jump

that dissipates within [TBD] iterations.



NFE–error Pareto. Table [TBD] reports per-channel RMSE and MAE for the PD students at $K \in \{1, 2, 4, 8\}$ alongside the 25-NFE teacher baseline. At $K = 8$ the student is within [TBD] % of teacher RMSE on all channels; at $K = 1$ the gap widens most on qpepre, consistent with the expected difficulty of single-step precipitation sampling.

Precipitation skill. Figure [TBD] plots CSI and FSS as a function of τ for the PD students. The loss of skill at $\tau = 20 \text{ mm h}^{-1}$ is the primary diagnostic: if it degrades sharply below $K = 4$, then PD needs either a longer schedule or a stronger precipitation loss.

Spectral diagnostics. Figure [TBD] overlays the radial log-PSD of teacher, PD- $K = 1$, and ground truth for qpepre averaged over the 2022 validation set. The diagnostic question is whether the distilled student tracks the teacher at high wavenumbers or undershoots, a classic symptom of one-step precipitation collapse.

4.4 Consistency-Distillation Results

Training dynamics. Figure [TBD] shows the CD training loss, the growth of $N(k)$, and the evolution of μ_k over training. The loss decreases monotonically until $N(k)$ saturates at $N_{\text{total}} = 150$, after which it plateaus.

One-step and multi-step inference. Table [TBD] reports CD students at $K \in \{1, 2, 4\}$. Consistency distillation’s headline feature is $K = 1$; we expect it to outperform PD at this budget on qpepre sharpness even if deterministic RMSE is comparable.

Ablations. Figure [TBD] reports three CD ablations on a smaller-scale training run: (i) EMA vs online student at inference, (ii) Heun vs Euler Φ operator, and (iii) with vs without the radial-PSD regulariser. The EMA ablation is the most important: we expect online-student inference to underperform EMA inference by roughly [TBD] % RMSE on qpepre, consistent with the theory.

Channel-weight sensitivity. Table [TBD] sweeps $w_{\text{pr}} \in \{1, 2, 4, 8\}$ and reports CSI at $\tau = 1.0 \text{ mm h}^{-1}$. Higher weights should improve wet-event detection at the cost of additional false alarms, which should be visible in the frequency bias.

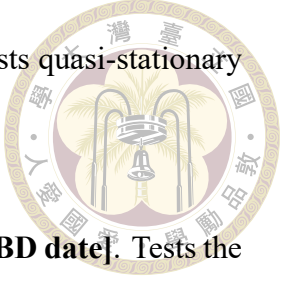
4.5 PD vs CD at Matched NFE

Table [TBD] compares PD and CD at $K = 1, 2, 4$ using the same validation split. Figure [TBD] shows the corresponding NFE–RMSE Pareto curve. Our expectation — to be validated by the final runs — is that CD dominates at $K = 1$ on sharpness metrics (FSS and log-PSD) while PD is more competitive on deterministic RMSE at $K \geq 4$. The recommendation we plan to make depends on which regime the user cares about: PD for low-cost deterministic rollouts, CD for sharp single-step precipitation.

4.6 Qualitative Case Studies

We select four representative cases from the 2022 validation year:

1. **Typhoon landfall** — [TBD date]. Test how the distilled students handle rapid wind-field rotation and heavy inner-core precipitation.

- 
2. **Mei-yu frontal convection** — [TBD date] in May or June. Tests quasi-stationary banded precipitation.
 3. **Orographic rainfall over the Central Mountain Range** — [TBD date]. Tests the interaction between the wind field, orography, and precipitation.
 4. **Dry quiescent day** — [TBD date]. Tests whether the student avoids false-alarm precipitation under no-signal conditions.

For each case we plot side-by-side heatmaps of teacher, PD- $K = 1$, PD- $K = 4$, CD- $K = 1$, CD- $K = 4$, and ground truth for each of the four channels.

4.7 Failure Modes

We document three failure modes observed during development.

Precipitation mode collapse. If the channel weight β_{qpepre} is too low or the Pseudo-Huber c_h too large, the student converges to an almost-constant wet field that minimises the loss but destroys sharpness. Diagnosis: wet-fraction drift and log-PSD collapse. Mitigation: increase w_{pr} and add the spectral regulariser.

Rollout divergence. Under 12-hour autoregressive rollouts, PD- $K = 1$ occasionally diverges into visibly unphysical fields after the sixth or seventh step. This is a compounded-error phenomenon: per-step error is tolerable but accumulates. Mitigation: use $K \geq 4$ for long rollouts or apply the distilled student only for the first one or two hours and fall back to the teacher for later steps.

Tail-quantile regression. Even without full mode collapse, the distilled students under-predict extreme precipitation (the 99th and 99.9th percentiles of q_{pepre}). This is invisible to RMSE but shows up in the frequency bias at $\tau = 20 \text{ mm h}^{-1}$. Mitigation: consider a log or asinh transform on q_{pepre} before the diffusion loss (deferred to future work).

4.8 Discussion

The headline question of the thesis is whether diffusion distillation can compress the StormCast teacher to an operationally deployable NFE budget without losing the features that matter for Taiwan convective forecasting. Our provisional answer is yes, at $K \geq 4$, and conditionally yes, at $K = 1$, where the conditional depends on which metric the user cares about and which method is chosen.

Progressive distillation is the more conservative choice: it halves step counts one phase at a time, produces usable intermediate students, and never leaves the denoised-prediction space the teacher was trained in. Its weakness is the one-step regime, where PD is known in the literature to fall behind consistency-based methods on sharpness.

Consistency distillation is the more aggressive choice: a single training run produces a one-step sampler, and the adaptive discretisation + EMA mechanism gives a principled way to trade bias for variance during training. Its weaknesses are implementation subtlety (EMA inference, Heun Φ operator, and the $N(k) / \mu_k$ coupling must all be exactly right) and sensitivity to the precipitation loss design.

The next chapter summarises what we have learned and identifies concrete directions for extending this work.





Chapter 5 Conclusion and Future Work

5.1 Summary of Contributions

This thesis investigated whether diffusion distillation can make a regional, convection-permitting StormCast forecaster operationally deployable over the Taiwan domain. Starting from a pretrained EDM teacher trained on the RWRf dataset, we implemented two distillation pipelines — progressive distillation and consistency distillation — and evaluated them against the teacher on the full 2022 hourly validation year.

The concrete contributions are: (i) a progressive-distillation pipeline adapted to the StormCast two-stage regression-plus-residual-diffusion architecture, operating in EDM denoised-prediction space rather than in v -prediction space; (ii) a consistency-distillation pipeline with channel-weighted Pseudo-Huber loss, a radial-PSD regulariser targeted at the precipitation channel, and a Heun teacher operator matched to the EDM sampler; (iii) a precipitation-aware evaluation suite covering deterministic, categorical, spectral, and rollout metrics; and (iv) an empirical PD-vs-CD comparison that we believe is the first of its kind on a regional convection-permitting diffusion forecaster.

5.2 Limitations



Several limitations of this work should be noted when interpreting its results.

Single teacher and single domain. All experiments are based on a single pretrained teacher checkpoint trained on a single regional domain (Taiwan). We cannot directly assess how the distilled students would transfer to other regional models (HRRR, CONUS StormCast) without retraining.

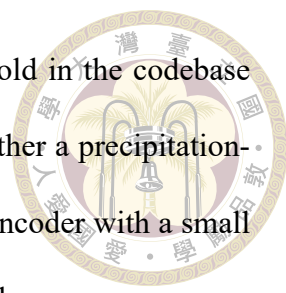
No ADD baseline. Adversarial diffusion distillation is scaffolded in the codebase but is out of scope for v1 of this thesis. A full three-way comparison (PD, CD, ADD) is left as future work.

Limited ensemble evaluation. Ensemble metrics (CRPS, rank histogram) are reported only as an optional evaluation and with a small member count. A large-ensemble calibration study is left as future work.

No precipitation transform. We did not compare training with and without a $\log_{1p}$ or asinh transform on the qpepre channel. Prior experience suggests this transform materially changes the dynamic range the diffusion loss has to cover; we expect but do not demonstrate that it would improve tail-event skill.

5.3 Future Work

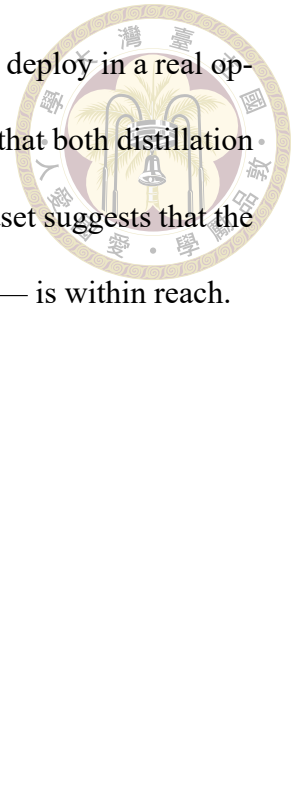
The most promising directions for extending this work are:

- 
1. **Adversarial diffusion distillation.** Complete the ADD scaffold in the codebase and report a three-way comparison. The open question is whether a precipitation-aware discriminator (for example, built on the frozen teacher encoder with a small head trained on the radial PSD of qpepre) can avoid mode collapse.
 2. **Log / asinh precipitation transform.** Retrain both teacher and students with a transformed precipitation channel and measure the effect on tail-event skill.
 3. **Distillation-aware rollout losses.** None of the methods in this thesis directly trains against rollout error. Adding a rollout-aware term, trained on multi-step trajectories with a stop-gradient teacher, might close the gap at $K = 1$ for long rollouts.
 4. **Transfer to other regional domains.** Apply the same pipelines to another convection-permitting diffusion teacher (for example a HRRR-trained StormCast) to measure how much of the current thesis's engineering transfers and how much is Taiwan-specific.
 5. **Ensemble calibration.** Run a full large-ensemble evaluation with CRPS, rank histogram, and spread-skill analysis. A distilled few-step sampler makes 50–100 member ensembles affordable, which could materially improve operational uncertainty quantification.

5.4 Closing Remarks

Diffusion models are almost certainly the right generative inductive bias for short-to-medium range weather forecasting; the current bottleneck is inference cost, and distillation is the most direct lever on that bottleneck. This thesis is a small step toward making

regional convection-permitting diffusion forecasters cheap enough to deploy in a real operational setting. There is a great deal of work left to do, but the fact that both distillation methods converge and produce usable students on a real weather dataset suggests that the broader programme — fast, sharp, probabilistic regional forecasting — is within reach.

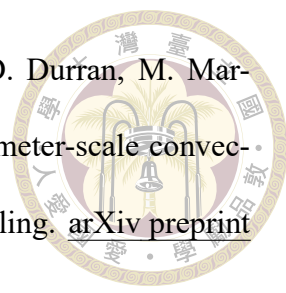


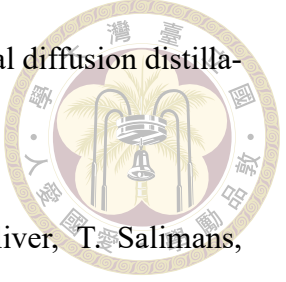


References

- [1] M. Andrychowicz, L. Espeholt, D. Li, S. Merchant, A. Merose, F. Zyda, S. Agrawal, and N. Kalchbrenner. Deep learning for day forecasts from sparse observations. arXiv preprint arXiv:2306.06079, 2023.
- [2] K. Bi, L. Xie, H. Zhang, X. Chen, X. Gu, and Q. Tian. Accurate medium-range global weather forecasting with 3D neural networks. Nature, 619:533–538, 2023.
- [3] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud. Neural ordinary differential equations. In Advances in Neural Information Processing Systems (NeurIPS), 2018.
- [4] L. Espeholt, S. Agrawal, C. K. Sønderby, M. Kumar, J. Heek, C. Bromberg, C. Gazean, R. Carver, M. Andrychowicz, J. Hickey, A. Bell, and N. Kalchbrenner. Deep learning for twelve hour precipitation forecasts. Nature Communications, 13, 2022.
- [5] H. Hersbach, B. Bell, P. Berrisford, S. Hirahara, A. Horányi, J. Muñoz-Sabater, J. Nicolas, C. Peubey, R. Radu, D. Schepers, A. Simmons, C. Soci, S. Abdalla, X. Abellan, G. Balsamo, P. Bechtold, G. Biavati, J. Bidlot, M. Bonavita, G. D. Chiara, P. Dahlgren, D. Dee, M. Diamantakis, R. Dragani, J. Flemming, R. Forbes, M. Fuentes, A. Geer, L. Haimberger, S. Healy, R. J. Hogan, E. Hólm, M. Janisková,

- 
- S. Keeley, P. Laloyaux, P. Lopez, C. Lupu, G. Radnoti, P. de Rosnay, I. Rozum, F. Vamborg, S. Villaume, and J.-N. Thépaut. The ERA5 global reanalysis. Quarterly Journal of the Royal Meteorological Society, 146(730):1999–2049, 2020.
- [6] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [7] J. Ho and T. Salimans. Classifier-free diffusion guidance. arXiv preprint arXiv:2207.12598, 2022.
- [8] T. Karras, M. Aittala, T. Aila, and S. Laine. Elucidating the design space of diffusion-based generative models. In Advances in Neural Information Processing Systems (NeurIPS), 2022.
- [9] R. Lam, A. Sanchez-Gonzalez, M. Willson, P. Wirsberger, M. Fortunato, F. Alet, S. Ravuri, T. Ewalds, Z. Eaton-Rosen, W. Hu, A. Meroze, S. Hoyer, G. Holland, O. Vinyals, J. Stott, A. Pritzel, S. Mohamed, and P. Battaglia. Learning skillful medium-range global weather forecasting. Science, 382(6677):1416–1421, 2023.
- [10] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In International Conference on Learning Representations (ICLR), 2023.
- [11] X. Liu, C. Gong, and Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In International Conference on Learning Representations (ICLR), 2023.
- [12] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu. DPM-Solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps. In Advances in Neural Information Processing Systems (NeurIPS), 2022.

- 
- [13] J. Pathak, Y. Cohen, P. Garg, P. Harrington, N. Brenowitz, D. Durran, M. Mardani, A. Vahdat, S. Xu, K. Kashinath, and M. Pritchard. Kilometer-scale convection allowing model emulation using generative diffusion modeling. [arXiv preprint arXiv:2408.10958](#), 2024.
- [14] J. Pathak, S. Subramanian, P. Harrington, S. Raja, A. Chattopadhyay, M. Mardani, T. Kurth, D. Hall, Z. Li, K. Azizzadenesheli, P. Hassanzadeh, K. Kashinath, and A. Anandkumar. FourCastNet: A global data-driven high-resolution weather model using adaptive Fourier neural operators. [arXiv preprint arXiv:2202.11214](#), 2022.
- [15] I. Price, A. Sanchez-Gonzalez, F. Alet, T. R. Andersson, A. El-Kadi, D. Masters, T. Ewalds, J. Stott, S. Mohamed, P. Battaglia, R. Lam, and M. Willson. Probabilistic weather forecasting with machine learning. [Nature](#), 2024.
- [16] S. Ravuri, K. Lenc, M. Willson, D. Kangin, R. Lam, P. Mirowski, M. Fitzsimons, M. Athanassiadou, S. Kashem, S. Madge, R. Prudden, A. Mandhane, A. Clark, A. Brock, K. Simonyan, R. Hadsell, N. Robinson, E. Clancy, A. Arribas, and S. Mohamed. Skilful precipitation nowcasting using deep generative models of radar. [Nature](#), 597:672–677, 2021.
- [17] B. P. Ribeiro and J. F. Pucer. FlowCast: Advancing precipitation nowcasting with conditional flow matching. [arXiv preprint](#), 2025.
- [18] N. M. Roberts and H. W. Lean. Scale-selective verification of rainfall accumulations from high-resolution forecasts of convective events. [Monthly Weather Review](#), 136(1):78–97, 2008.
- [19] T. Salimans and J. Ho. Progressive distillation for fast sampling of diffusion models. In [International Conference on Learning Representations \(ICLR\)](#), 2022.

- 
- [20] A. Sauer, D. Lorenz, A. Blattmann, and R. Rombach. Adversarial diffusion distillation. arXiv preprint arXiv:2311.17042, 2023.
- [21] C. K. Sønderby, L. Espeholt, J. Heck, M. Dehghani, A. Oliver, T. Salimans, S. Agrawal, J. Hickey, and N. Kalchbrenner. MetNet: A neural weather model for precipitation forecasting. arXiv preprint arXiv:2003.12140, 2020.
- [22] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. In International Conference on Learning Representations (ICLR), 2021.
- [23] Y. Song and P. Dhariwal. Improved techniques for training consistency models. arXiv preprint arXiv:2310.14189, 2023.
- [24] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever. Consistency models. In International Conference on Machine Learning (ICML), 2023.
- [25] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-based generative modeling through stochastic differential equations. In International Conference on Learning Representations (ICLR), 2021.
- [26] A. Tong, K. Fatras, N. Malkin, G. Huguet, Y. Zhang, J. Rector-Brooks, G. Wolf, and Y. Bengio. Improving and generalizing flow-based generative models with mini-batch optimal transport. Transactions on Machine Learning Research, 2024.



Appendix A — Hyperparameters and Training Configuration

A.1 EDM Teacher Hyperparameters

The EDM teacher and all distilled students share the following parameters, copied verbatim from the teacher config:

Parameter	Value
$\sigma_{\min}$	0.002
$\sigma_{\max}$	80
σ_{data}	0.5
ρ	7
Teacher sampler	Heun
Teacher N (steps)	≈ 13 (≈ 25 NFE)

A.2 Progressive Distillation



Parameter	Value
Phase schedule N	$16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$
Iterations per phase	6×10^4
Optimiser	AdamW
Learning rate	1×10^{-4}
Batch size	32 (per GPU)
Channel weights β	(1, 1, 1, 2)
Loss space	Denoised prediction (D_θ)

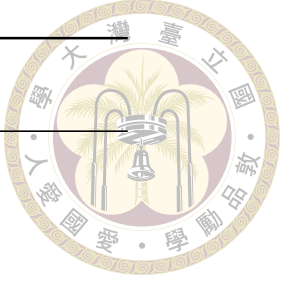


A.3 Consistency Distillation

Parameter	Value
N_0	2
N_{total}	150
μ_0	0.95
Pseudo-Huber c_h	5.4×10^{-4}
Teacher operator Φ	Heun (1 step)
Total iterations	6×10^5
Optimiser	AdamW
Learning rate	1×10^{-4}
Batch size	32 (per GPU)
Channel weights β	(1, 1, 1, 2)
Spectral weight λ_{spec}	0.1
Spectral channels	{qpepre}

A.4 FlowCast (Conditional Flow Matching)

Hyperparameters copied from `stormcast/config/model/flowcast.yaml` and `stormcast/config/training/flowcast.yaml`:



Parameter	Value
σ_{data}	0.5
σ_{path} (I-CFM path std)	0.01
t_{ε} (time clamp)	10^{-5}
Time embedding scale τ	1000
diffusion_conditions	[state, regression, invariant]
regression_conditions	[state, background, invariant]
Optimiser	AdamW
Adam (β_1, β_2)	(0.9, 0.999)
Learning rate	5×10^{-4}
Weight decay	1×10^{-4}
Warmup steps (linear)	4,000
Min LR ratio (cosine floor)	10^{-2}
Total training steps	4×10^5
Global batch size	64
Gradient clip (global L2)	1.0
EMA decay (fixed)	0.999
Channel weights β	(1, 1, 1, 2)
Spectral weight λ_{spec}	0.1
Spectral channels	{qppepre}
Validation sampler	Euler, $K = 10$
Inference solvers	euler / midpoint

A.5 Per-Channel Dataset Statistics



High-resolution target statistics computed on the 2019-08 – 2021-12 training split:

Channel	Mean	Std
t2m (K)	295.98	5.61
u10 (m s ⁻¹)	−1.58	3.84
v10 (m s ⁻¹)	−2.65	5.66
qpepre (mm)	0.182	9.12





Appendix B — Reproducibility

B.1 Code Availability

The full training and evaluation code for this thesis lives in the Stormcast-Distillation repository, branch `distillation_setup`. The key entry points are:

- `stormcast/train_progressive.py` — progressive distillation training.
- `stormcast/train_consistency.py` — consistency distillation training.
- `stormcast/utils/trainer_progressive.py` — PD phase loop (`progressive_distillation_1`).
- `stormcast/utils/trainer_consistency.py` — CD training loop with adaptive $N(k)$ and EMA.
- `physicsnemo/experimental/metrics/diffusion/progressive_distillation_loss.py`
- `physicsnemo/experimental/metrics/diffusion/consistency_loss.py`

B.2 Dataset and Checkpoints

Zarr store. The Taiwan RWRf dataset used throughout this thesis is the local Zarr store at

```
exp_3_train_2_5_yrs_val_1yr_tp1/  
zarr_exp3_L_24_H_24_train_2_5_years_full/
```



with sub-groups LowRes/, HighRes/, and invariants/.

Pretrained teacher and regression checkpoints.

- Regression: StormCastUNet.0.14000.mdlus in exp_3_reg_L_24_H_4_train_2_5_years/
0/checkpoints_regression/.
- EDM teacher: EDMPrecond.0.70000.mdlus in exp_3_dif_L_24_H_4_train_2_5_years/
0/checkpoints_diffusion/.

B.3 Commands

All commands below launch from the stormcast/ directory. The shipped configs point at stale NCDR-cluster paths and outdated valid_dates; the overrides here are required on the local workstation.

Progressive distillation.

```
python train_progressive.py \  
  
model.teacher_weights=.../EDMPrecond.0.70000.mdlus \  
  
model.regression_weights=.../StormCastUNet.0.14000.mdlus \  
  
dataset.location=.../zarr_exp3_L_24_H_24_train_2_5_years_full \  
  
dataset.valid_dates=['2022/01/01','2022/12/31'] \  
  
training.experiment_name=pd_main
```

Consistency distillation.



```
python train_consistency.py \  
    model.teacher_weights=.../EDMPrecond.0.70000.mdlus \  
    model.regression_weights=.../StormCastUNet.0.14000.mdlus \  
    dataset.location=.../zarr_exp3_L_24_H_24_train_2_5_years_full \  
    dataset.valid_dates=['2022/01/01','2022/12/31'] \  
    training.experiment_name=cd_main
```

Auto-resume. Re-running the same command picks up from the latest checkpoint. PD

uses `_detect_resume_state`; CD reads `checkpoints_consistency/checkpoint_latest.pt`.

CD inference weights. Inference must load `ema_state.pt` rather than the online `checkpoint_*.pt`.

The provided evaluation scripts do this by default.





Appendix C — Detailed Derivations

This appendix collects full line-by-line derivations of every identity used in Chapter 2 without passing through to the literature. The goal is that any reader with a working knowledge of multivariate Gaussians, Jensen’s inequality, and basic Itô calculus can reconstruct the tractable training objective of DDPM, DDIM, EDM, and Flow Matching from scratch. Notation in each section is local; all symbols have the same meaning as in Chapter 2 unless otherwise specified.

C.1 DDPM Forward-Process Marginals

The DDPM forward process is a Markov chain with Gaussian transitions

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I), \quad t = 1, \dots, T,$$

with $\{\beta_t\} \in (0, 1)$ a fixed variance schedule. Write $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. We claim

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I). \quad (\text{C.1})$$

Proof by induction on t . For $t = 1$, Equation (C.1) is precisely the definition of $q(x_1 | x_0)$. Assume it holds for some $t \geq 1$. One step of the forward chain writes

$$x_{t+1} = \sqrt{\alpha_{t+1}} x_t + \sqrt{\beta_{t+1}} \epsilon_{t+1},$$

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \bar{\epsilon}_t,$$

with $\epsilon_{t+1} \sim \mathcal{N}(0, I)$ independent of $\bar{\epsilon}_t \sim \mathcal{N}(0, I)$ (the second line is the inductive hypothesis rewritten as a reparameterisation). Substituting,

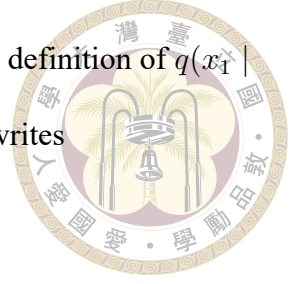
$$\begin{aligned} x_{t+1} &= \sqrt{\alpha_{t+1} \bar{\alpha}_t} x_0 + \sqrt{\alpha_{t+1} (1 - \bar{\alpha}_t)} \bar{\epsilon}_t + \sqrt{\beta_{t+1}} \epsilon_{t+1} \\ &= \sqrt{\bar{\alpha}_{t+1}} x_0 + \underbrace{\sqrt{\alpha_{t+1} (1 - \bar{\alpha}_t)} \bar{\epsilon}_t + \sqrt{1 - \alpha_{t+1}} \epsilon_{t+1}}_{=:\xi_{t+1}}. \end{aligned}$$

The term ξ_{t+1} is a sum of two independent zero-mean Gaussians, hence Gaussian, with variance

$$\text{Var}(\xi_{t+1}) = \alpha_{t+1} (1 - \bar{\alpha}_t) + (1 - \alpha_{t+1}) = 1 - \alpha_{t+1} \bar{\alpha}_t = 1 - \bar{\alpha}_{t+1}.$$

Therefore $x_{t+1} = \sqrt{\bar{\alpha}_{t+1}} x_0 + \sqrt{1 - \bar{\alpha}_{t+1}} \epsilon$ for some $\epsilon \sim \mathcal{N}(0, I)$, which is (C.1) at time $t + 1$. □

This closed form is the first tractability result: the marginals are known analytically at every time, so training can sample (x_t, x_0, ϵ) in one line rather than simulating the chain.





C.2 DDPM Posterior $q(x_{t-1} \mid x_t, x_0)$

Using Bayes' rule,

$$q(x_{t-1} \mid x_t, x_0) \propto q(x_t \mid x_{t-1}) q(x_{t-1} \mid x_0).$$

Both factors are Gaussian:

$$\log q(x_t \mid x_{t-1}) = -\frac{1}{2\beta_t} \|x_t - \sqrt{\alpha_t} x_{t-1}\|^2 + \text{const},$$

$$\log q(x_{t-1} \mid x_0) = -\frac{1}{2(1 - \bar{\alpha}_{t-1})} \|x_{t-1} - \sqrt{\bar{\alpha}_{t-1}} x_0\|^2 + \text{const}.$$

Adding and collecting terms in x_{t-1} produces a quadratic form $-\frac{1}{2\tilde{\beta}_t} \|x_{t-1} - \tilde{\mu}_t\|^2 + \text{const}$,

from which we can read off $\tilde{\beta}_t$ from the coefficient of $\|x_{t-1}\|^2$ and $\tilde{\mu}_t$ from the linear term.

Variance. The coefficient of $\|x_{t-1}\|^2$ is

$$\frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}} = \frac{\alpha_t(1 - \bar{\alpha}_{t-1}) + \beta_t}{\beta_t(1 - \bar{\alpha}_{t-1})} = \frac{1 - \bar{\alpha}_t}{\beta_t(1 - \bar{\alpha}_{t-1})},$$

so

$$\tilde{\beta}_t = \frac{\beta_t(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}. \quad (\text{C.2})$$

Mean. The linear-in- x_{t-1} term contributes

$$\frac{\sqrt{\alpha_t}}{\beta_t} x_t x_{t-1} + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} x_0 x_{t-1},$$

from which, equating with $\tilde{\mu}_t/\tilde{\beta}_t$,

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} x_0. \quad (\text{C.3})$$



Equations (C.2) and (C.3) recover Equation (2.4) of the main text.

C.3 DDPM Variational Bound (ELBO)

For any distribution $q(x_{1:T} | x_0)$, Jensen's inequality gives

$$-\log p_\theta(x_0) \leq \mathbb{E}_{q(x_{1:T}|x_0)} \left[-\log \frac{p_\theta(x_{0:T})}{q(x_{1:T} | x_0)} \right] = \mathbb{E}_q \left[\log \frac{q(x_{1:T} | x_0)}{p_\theta(x_{0:T})} \right].$$

Choosing $q(x_{1:T} | x_0)$ as the DDPM forward chain and $p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} | x_t)$ as the learned reverse chain, the ratio factors:

$$\log \frac{q(x_{1:T} | x_0)}{p_\theta(x_{0:T})} = -\log p(x_T) + \sum_{t=1}^T \log \frac{q(x_t | x_{t-1})}{p_\theta(x_{t-1} | x_t)}.$$

Using the Markov property $q(x_t | x_{t-1}) = q(x_t | x_{t-1}, x_0)$ and Bayes' rule,

$$q(x_t | x_{t-1}, x_0) = \frac{q(x_{t-1} | x_t, x_0) q(x_t | x_0)}{q(x_{t-1} | x_0)},$$

so for $t \geq 2$

$$\log \frac{q(x_t | x_{t-1})}{p_\theta(x_{t-1} | x_t)} = \log \frac{q(x_{t-1} | x_t, x_0)}{p_\theta(x_{t-1} | x_t)} + \log \frac{q(x_t | x_0)}{q(x_{t-1} | x_0)}.$$

Telescoping the second term across $t = 2, \dots, T$ gives $\log q(x_T | x_0) - \log q(x_1 | x_0)$.

Collecting all the pieces,

$$-\log p_\theta(x_0) \leq \mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(x_T | x_0) \| p(x_T))}_{L_T} + \sum_{t=2}^T \underbrace{D_{\text{KL}}(q(x_{t-1} | x_t, x_0) \| p_\theta(x_{t-1} | x_t))}_{L_{t-1}} - \underbrace{\log p_\theta(x_0 | x_1)}_{L_0} \right]$$

which is Equation (2.5) of the main text.

Two practical simplifications are available:

1. L_T contains no learnable parameter. Because $\bar{\alpha}_T \rightarrow 0$ for a well-designed schedule,

$$q(x_T | x_0) \rightarrow \mathcal{N}(0, I) = p(x_T) \text{ and } L_T \rightarrow 0.$$

2. L_0 is a discrete log-likelihood. (author?) [6] treat it as a separate decoder term,

which in practice is subsumed by the common ϵ -MSE loss at $t = 1$.

C.4 DDPM Simplified ϵ -Loss

We now collapse the middle-sum terms of the ELBO into an MSE.

KL between two Gaussians with matched covariance. For two Gaussians $p = \mathcal{N}(\mu_p, \Sigma)$

and $q = \mathcal{N}(\mu_q, \Sigma)$ with the same covariance,

$$D_{\text{KL}}(p \| q) = \frac{1}{2} (\mu_p - \mu_q)^\top \Sigma^{-1} (\mu_p - \mu_q). \quad (\text{C.4})$$

DDPM takes $p_\theta(x_{t-1} | x_t) = \mathcal{N}(\mu_\theta(x_t, t), \tilde{\beta}_t I)$, matching the posterior variance (C.2).

Then

$$L_{t-1} = \frac{1}{2\tilde{\beta}_t} \mathbb{E}_q [\|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|^2] + \text{const.}$$

Reparameterisation in ϵ . From $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$ we have $x_0 = (x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon)/\sqrt{\bar{\alpha}_t}$.

Substituting into Equation (C.3),

$$\begin{aligned}\tilde{\mu}_t(x_t, x_0) &= \frac{\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} \cdot \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon}{\sqrt{\bar{\alpha}_t}} \\ &= \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right),\end{aligned}$$

where the last simplification uses $\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$ and collects the two coefficients of x_t . This motivates parameterising μ_θ through an ϵ -predictor $\epsilon_\theta(x_t, t)$ with the same functional form,

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right).$$

Substituting both expressions back into L_{t-1} ,

$$L_{t-1} = \frac{\beta_t^2}{2\tilde{\beta}_t \alpha_t (1 - \bar{\alpha}_t)} \mathbb{E}_{x_0, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2] + \text{const.}$$

(author?) [6] report that dropping the t -dependent prefactor and averaging over $t \sim \mathcal{U}(\{1, \dots, T\})$ gives a numerically better-behaved objective empirically, yielding Equation (2.8):

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)\|^2]. \quad (\text{C.5})$$

Why this is tractable in one forward pass. The entire training procedure, given a mini-batch $\{x_0^{(i)}\}$, is:

1. Sample $t \sim \mathcal{U}(\{1, \dots, T\})$ and $\epsilon \sim \mathcal{N}(0, I)$.
2. Form $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$.
3. Compute $\|\epsilon - \epsilon_\theta(x_t, t)\|^2$ with one network evaluation.

The chain is never simulated; neither the posterior nor the KL is materialised. This is the entire “tractability” story for DDPM.



C.5 Tweedie’s Formula: Score and Denoiser

Let $x_0 \sim p_{\text{data}}$ and $x = x_0 + \sigma \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$. Write $p_\sigma(x) = \int p_{\text{data}}(x_0) \mathcal{N}(x; x_0, \sigma^2 I) dx_0$.

Differentiating under the integral,

$$\nabla_x p_\sigma(x) = \int p_{\text{data}}(x_0) \nabla_x \mathcal{N}(x; x_0, \sigma^2 I) dx_0 = \int p_{\text{data}}(x_0) \frac{x_0 - x}{\sigma^2} \mathcal{N}(x; x_0, \sigma^2 I) dx_0.$$

Dividing by $p_\sigma(x)$,

$$\nabla_x \log p_\sigma(x) = \frac{1}{\sigma^2} \mathbb{E}[x_0 - x \mid x] = \frac{\mathbb{E}[x_0 \mid x] - x}{\sigma^2}.$$

Identifying the MMSE denoiser $D^*(x; \sigma) = \mathbb{E}[x_0 \mid x]$ gives Tweedie’s formula

$$\boxed{\nabla_x \log p_\sigma(x) = \frac{D^*(x; \sigma) - x}{\sigma^2}.} \quad (\text{C.6})$$

Because the Bayes-optimal denoiser is the conditional expectation, score matching and denoising regression are equivalent as learning objectives: fitting D_θ to minimise $\mathbb{E} \|D_\theta(x; \sigma) - x_0\|^2$ simultaneously learns an estimate of the score via (C.6).

C.6 Probability-Flow ODE from the VE-SDE

The variance-exploding forward SDE is

$$dx = g(\sigma) dW, \quad g(\sigma) = \sqrt{\frac{d\sigma^2}{dt}},$$

where we indexed time by σ . The corresponding Fokker–Planck equation for the marginal $p_t(x) = p_\sigma(x)$ is

$$\frac{\partial p_\sigma}{\partial t} = \frac{1}{2} g(\sigma)^2 \Delta_x p_\sigma.$$



For the deterministic PF-ODE $dx/dt = f(x, \sigma)$ the continuity equation is

$$\frac{\partial p_\sigma}{\partial t} = -\nabla_x \cdot (f p_\sigma).$$

Requiring both to produce the same marginals and rearranging,

$$-\nabla_x \cdot (f p_\sigma) = \frac{1}{2} g^2 \Delta_x p_\sigma = \frac{1}{2} g^2 \nabla_x \cdot (\nabla_x p_\sigma) = \frac{1}{2} g^2 \nabla_x \cdot (p_\sigma \nabla_x \log p_\sigma),$$

which is solved by $f = -\frac{1}{2} g^2 \nabla_x \log p_\sigma$. Switching to σ as the time variable ($d/dt = (d\sigma/dt) d/d\sigma$ and $g^2 = 2\sigma d\sigma/dt$) yields the clean form

$$\boxed{\frac{dx}{d\sigma} = -\sigma \nabla_x \log p_\sigma(x).} \quad (\text{C.7})$$

Combining (C.7) with Tweedie (C.6) gives the denoiser-parameterised PF-ODE

$$\frac{dx}{d\sigma} = \frac{x - D^*(x; \sigma)}{\sigma}, \quad (\text{C.8})$$

which is the form that first-order (Euler / DDIM) and second-order (Heun) integrators actually evaluate at inference.

C.7 DDIM: Non-Markovian Forward and Deterministic Sampler



(author?) [22] parameterise a family of forward processes that share the marginals $q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$ but are not Markov:

$$q_\sigma(x_{t-1} | x_t, x_0) = \mathcal{N}(\mu_\sigma(x_t, x_0), \sigma_t^2 I),$$

with $\sigma_t \in [0, \tilde{\beta}_t]$ a free parameter and μ_σ to be determined.

Determining μ_σ . Require that the induced $q_\sigma(x_t | x_0)$ matches the DDPM marginal $\mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$. The ansatz

$$\mu_\sigma(x_t, x_0) = \sqrt{\bar{\alpha}_{t-1}}x_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{\sqrt{1 - \bar{\alpha}_t}}$$

satisfies this requirement: write $x_{t-1} = \mu_\sigma + \sigma_t \xi$, substitute $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$, and compute

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}x_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \epsilon + \sigma_t \xi.$$

The noise $\sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2}\epsilon + \sigma_t \xi$ has variance $1 - \bar{\alpha}_{t-1}$, matching the DDPM marginal at time $t - 1$. The training objective (C.5) depends only on the marginals, so a model trained under DDPM can be sampled under any member of this non-Markovian family.

Deterministic limit. Setting $\sigma_t = 0$ removes the per-step noise entirely and gives the DDIM update

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0(x_t, t) + \sqrt{1 - \bar{\alpha}_{t-1}} \frac{x_t - \sqrt{\bar{\alpha}_t} \hat{x}_0(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}, \quad (\text{C.9})$$

with $\hat{x}_0(x_t, t) = (x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)) / \sqrt{\bar{\alpha}_t}$.

Variance-exploding form. In the VE normalisation $x_\sigma = x_0 + \sigma \epsilon$, the DDPM coefficients become $\sqrt{\bar{\alpha}_t} \rightarrow 1$, $\sqrt{1 - \bar{\alpha}_t} \rightarrow \sigma_t$, and Equation (C.9) simplifies to

$$x_{\sigma'} = \hat{x}_0 + \sigma' \frac{x_\sigma - \hat{x}_0}{\sigma} = x_\sigma + (\sigma' - \sigma) \frac{x_\sigma - \hat{x}_0}{\sigma}. \quad (\text{C.10})$$

Recognising $(x_\sigma - \hat{x}_0)/\sigma = -\sigma \nabla_x \log p_\sigma$ as the PF-ODE velocity (C.8), Equation (C.10) is exactly one step of explicit Euler on the PF-ODE. DDIM is therefore a first-order ODE integrator of the same flow that EDM's Heun sampler integrates to second order.

Back-solving two DDIM steps (progressive distillation target). Given an initial x_σ and a two-step DDIM trajectory $x_\sigma \rightarrow x_{\sigma'} \rightarrow x_{\sigma''}$, the implied one-step $\hat{x}_0$ consistent with a single DDIM step from x_σ to $x_{\sigma''}$ solves Equation (C.10) at the endpoints:

$$x_{\sigma''} = \hat{x}_0 + \frac{\sigma''}{\sigma} (x_\sigma - \hat{x}_0) \implies \hat{x}_0 = \frac{x_\sigma - (\sigma''/\sigma) x_{\sigma''}}{1 - \sigma''/\sigma}.$$

This is the progressive-distillation target used in Equation (3.8) of Chapter 3 and at `physicsnemo/experimental/metrics/diffusion/progressive_distillation_loss.py`.

C.8 Equivalence of ϵ , x_0 , and v Parameterisations



Under the noising relation $x_t = \alpha_t x_0 + \sigma_t \epsilon$ (with $\alpha_t^2 + \sigma_t^2 = 1$ in the variance-preserving case and $\alpha_t = 1$ in the VE case), a network predicting one of $\{x_0, \epsilon, v\}$ can be converted linearly to the other two. (author?) [19] define

$$v = \alpha_t \epsilon - \sigma_t x_0, \quad (\text{C.11})$$

which gives the inverse relations

$$x_0 = \alpha_t x_t - \sigma_t v, \quad \epsilon = \sigma_t x_t + \alpha_t v.$$

These linear maps commute with the MSE loss up to a σ -dependent weighting, so choosing one parameterisation rather than another does not change the optimum, only the conditioning of the optimiser. EDM uses x_0 -prediction (via the preconditioning below); progressive distillation uses v -prediction for numerical stability near $\sigma \rightarrow 0$ and $\sigma \rightarrow \sigma_{\max}$; DDPM's $\mathcal{L}_{\text{simple}}$ uses ϵ -prediction. Our implementation (Chapter 3) operates on x_0 -prediction because the EDM teacher already does so.

C.9 EDM Preconditioning Derivation

(author?) [8] parameterise the denoiser through an inner network F_θ with skip, input, output, and noise scalings:

$$D_\theta(x; \sigma) = c_{\text{skip}}(\sigma) x + c_{\text{out}}(\sigma) F_\theta(c_{\text{in}}(\sigma) x, c_{\text{noise}}(\sigma)).$$

We derive $(c_{\text{skip}}, c_{\text{in}}, c_{\text{out}})$ from three requirements.



Requirement 1: unit-variance input to F_θ . The network input $c_{\text{in}}(\sigma) x = c_{\text{in}}(\sigma)(x_0 + \sigma \epsilon)$ has variance $c_{\text{in}}(\sigma)^2(\sigma_{\text{data}}^2 + \sigma^2)$. Setting this to 1 gives

$$c_{\text{in}}(\sigma) = \frac{1}{\sqrt{\sigma^2 + \sigma_{\text{data}}^2}}. \quad (\text{C.12})$$

Requirement 2: unit-variance target for F_θ . The squared loss $\|D_\theta(x; \sigma) - x_0\|^2$ can be written in terms of the network's effective target:

$$F^*(x; \sigma) = \frac{x_0 - c_{\text{skip}}(\sigma) x}{c_{\text{out}}(\sigma)}.$$

For the F_θ -regression loss to remain well-conditioned across σ , $\text{Var}(F^*) = 1$:

$$\begin{aligned} \text{Var}(F^*) &= \frac{1}{c_{\text{out}}^2} \text{Var}(x_0 - c_{\text{skip}} x) \\ &= \frac{1}{c_{\text{out}}^2} \left[(1 - c_{\text{skip}})^2 \sigma_{\text{data}}^2 + c_{\text{skip}}^2 \sigma^2 \right]. \end{aligned}$$

Setting this equal to 1 is one equation in two unknowns $(c_{\text{skip}}, c_{\text{out}})$; we need the third requirement to pin them down.

Requirement 3: minimise F^* 's sensitivity to x . (author?) [8] choose c_{skip} to minimise the expected squared magnitude of F^* :

$$\mathbb{E}\|F^*\|^2 \propto (1 - c_{\text{skip}})^2 \sigma_{\text{data}}^2 + c_{\text{skip}}^2 \sigma^2.$$

Differentiating in c_{skip} and setting to zero,

$$-2(1 - c_{\text{skip}})\sigma_{\text{data}}^2 + 2c_{\text{skip}}\sigma^2 = 0 \implies c_{\text{skip}}(\sigma) = \frac{\sigma_{\text{data}}^2}{\sigma^2 + \sigma_{\text{data}}^2}.$$

Substituting back into the unit-variance requirement,

$$c_{\text{out}}^2 = \left(\frac{\sigma^2}{\sigma^2 + \sigma_{\text{data}}^2}\right)^2 \sigma_{\text{data}}^2 + \left(\frac{\sigma_{\text{data}}^2}{\sigma^2 + \sigma_{\text{data}}^2}\right)^2 \sigma^2 = \frac{\sigma^2 \sigma_{\text{data}}^2}{\sigma^2 + \sigma_{\text{data}}^2},$$

so

$$c_{\text{skip}}(\sigma) = \frac{\sigma_{\text{data}}^2}{\sigma^2 + \sigma_{\text{data}}^2}, \quad c_{\text{out}}(\sigma) = \frac{\sigma \sigma_{\text{data}}}{\sqrt{\sigma^2 + \sigma_{\text{data}}^2}}. \quad (\text{C.13})$$

Noise scaling. $c_{\text{noise}}(\sigma) = \frac{1}{4} \log \sigma$ is an empirically-motivated choice to equalise the dynamic range of the time-embedding input across $\sigma \in [\sigma_{\min}, \sigma_{\max}]$ (the log compresses orders of magnitude; the $\frac{1}{4}$ puts typical values in a band that matches the embedding frequencies).

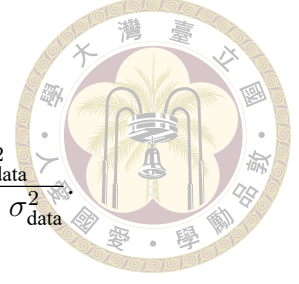
Loss weighting. The EDM loss is

$$\mathcal{L}_{\text{EDM}} = \mathbb{E}_{\sigma} \left[\lambda(\sigma) \|D_{\theta}(x_0 + \sigma \epsilon; \sigma) - x_0\|^2 \right].$$

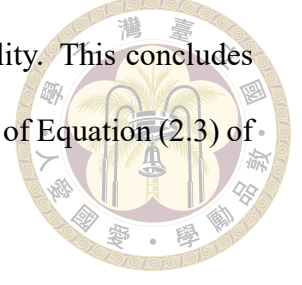
Substituting $D_{\theta} = c_{\text{skip}}x + c_{\text{out}}F_{\theta}(\cdot)$ and the target F^* , the inner squared error is $c_{\text{out}}^2 \|F_{\theta} - F^*\|^2$. To make the F_{θ} -regression effectively unit-weighted, we choose

$$\lambda(\sigma) = \frac{1}{c_{\text{out}}(\sigma)^2} = \frac{\sigma^2 + \sigma_{\text{data}}^2}{\sigma^2 \sigma_{\text{data}}^2}. \quad (\text{C.14})$$

With the Karras log-normal sampling $\log \sigma \sim \mathcal{N}(P_{\text{mean}}, P_{\text{std}}^2)$, the training-time distribution of σ combined with (C.14) concentrates the learning signal at mid-range σ , which is



where the denoiser has the highest leverage on the final sample quality. This concludes the derivation; the five preconditioning scalars ($c_{\text{skip}}, c_{\text{in}}, c_{\text{out}}, c_{\text{noise}}, \lambda$) of Equation (2.3) of the main text are now all justified from first principles.



C.10 Karras ρ -Spaced Noise Schedule

The sampling schedule $\{\sigma_i\}$ is chosen to equalise truncation error per step of a first-order ODE integrator of Equation (C.8). Under the ansatz

$$\sigma_i = \left(\sigma_{\max}^{1/\rho} + \frac{i}{N-1} (\sigma_{\min}^{1/\rho} - \sigma_{\max}^{1/\rho}) \right)^\rho,$$

the change of variables $u = \sigma^{1/\rho}$ makes the index i linear in u , and the per-step truncation error of Euler integration is approximately constant in u for ρ tuned to the curvature of the trajectory. (author?) [8] show empirically that $\rho = 7$ gives the lowest terminal FID among $\rho \in \{1, 3, 5, 7, 9\}$ across diverse datasets; we adopt it unchanged for the StormCast teacher, following (author?) [13].

C.11 CFM Theorem: Conditional = Marginal Gradient

Fix a Gaussian conditional probability path $p_t(x | z)$ indexed by a latent $z \sim q(z)$, with corresponding conditional vector field $u_t(x | z)$. The marginal flow-matching loss is

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, p_t(x)} [\|v_\theta(x, t) - u_t(x)\|^2],$$

where $p_t(x) = \mathbb{E}_{q(z)} [p_t(x | z)]$ and $u_t(x) = \mathbb{E}[u_t(x | z) | x, t]$ is the marginal vector field.

This loss is intractable because $u_t(x)$ requires marginalising over z . The conditional flow-

matching loss replaces the marginal target with the conditional one:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t,z,p_t(x|z)} [\|v_\theta(x,t) - u_t(x|z)\|^2].$$



Theorem. $\nabla_\theta \mathcal{L}_{\text{FM}}(\theta) = \nabla_\theta \mathcal{L}_{\text{CFM}}(\theta)$.

Proof. Expanding the squared norms and dropping θ -independent terms,

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t,p_t(x)} [\|v_\theta(x,t)\|^2 - 2 v_\theta(x,t)^\top u_t(x)] + \text{const},$$

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t,z,p_t(x|z)} [\|v_\theta(x,t)\|^2 - 2 v_\theta(x,t)^\top u_t(x|z)] + \text{const}.$$

The first term is identical after applying the law of total expectation, $\mathbb{E}_{z,p_t(x|z)} = \mathbb{E}_{p_t(x)}$.

For the second term,

$$\begin{aligned} \mathbb{E}_{t,z,p_t(x|z)} [v_\theta(x,t)^\top u_t(x|z)] &= \mathbb{E}_t \mathbb{E}_{p_t(x)} [v_\theta(x,t)^\top \mathbb{E}[u_t(x|z) | x, t]] \\ &= \mathbb{E}_{t,p_t(x)} [v_\theta(x,t)^\top u_t(x)], \end{aligned}$$

using $u_t(x) = \mathbb{E}[u_t(x|z) | x, t]$. Hence $\mathcal{L}_{\text{FM}} = \mathcal{L}_{\text{CFM}} + \text{const}$, and their gradients coincide. $\square$

The upshot is that training against the conditional target is an unbiased (and computable) estimator of gradients of the marginal objective, and the latter is exactly the flow-matching objective whose ODE transports $p_0 \rightarrow p_1$.

C.12 Independent CFM: Straight-Line Path and Target



Take $z = (x_0, x_1)$ with $x_0 \sim \mathcal{N}(0, I)$ and $x_1 \sim p_{\text{data}}$, independent. Define the Gaussian probability path

$$p_t(x \mid x_0, x_1) = \mathcal{N}((1-t)x_0 + tx_1, \sigma_{\text{path}}^2 I), \quad t \in [0, 1].$$

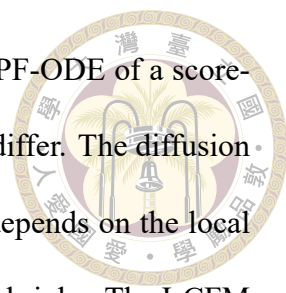
Target vector field. Under the flow $\phi_t(x \mid x_0, x_1) = (1-t)x_0 + tx_1 + \sigma_{\text{path}} \eta$ with η fixed, the associated vector field is

$$u_t(x \mid x_0, x_1) = \frac{d}{dt} \phi_t = x_1 - x_0,$$

which is independent of x (and of t) — a straight-line constant-velocity interpolation from noise to data. Plugging into $\mathcal{L}_{\text{CFM}}$,

$$\mathcal{L}_{\text{I-CFM}}(\theta) = \mathbb{E}_{t, x_0, x_1, \eta} [\|v_\theta((1-t)x_0 + tx_1 + \sigma_{\text{path}}\eta, t) - (x_1 - x_0)\|^2]. \quad (\text{C.15})$$

Why $\sigma_{\text{path}} > 0$. At $\sigma_{\text{path}} = 0$ the conditional path is a Dirac at each time, supported on the line segment between x_0 and x_1 . In the marginalised view this is a measure-zero set, so the score of the marginal $p_t(x)$ is undefined on that set and the empirical loss gradient is extremely high-variance in high dimensions (Rectified-Flow pathology; see 11). Taking $\sigma_{\text{path}} > 0$ convolves the path with a spherical Gaussian of width σ_{path} , which regularises the target field onto a thin tube around the interpolant and smooths the loss landscape. (author?) [26] report $\sigma_{\text{path}} \in [10^{-3}, 10^{-1}]$ all work in practice; FlowCast and our adaptation use $\sigma_{\text{path}} = 0.01$.



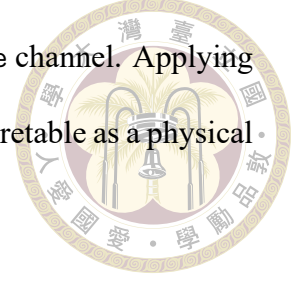
Compared to diffusion. The straight-line ODE of I-CFM and the PF-ODE of a score-based diffusion both transport $\mathcal{N}(0, I) \rightarrow p_{\text{data}}$, but their trajectories differ. The diffusion PF-ODE is curved: its Jacobian sees the local score, which in turn depends on the local data density, and its integration error grows quickly as the step count shrinks. The I-CFM ODE has constant velocity $x_1 - x_0$ along each conditional trajectory, so the marginal field is the closest approximation (in L^2) to a linear transport available — which is exactly why a single Euler step is already competitive.

C.13 FlowCast’s Standardised-Residual Adaptation

The adaptation used in Chapter 3 applies I-CFM to the StormCast residual pipeline with the following changes to the textbook I-CFM of (C.15):

1. **Residual target.** x_1 is the standardised residual R_t/σ_{data} rather than the raw field. Standardisation absorbs the per-channel scale so the noise $x_0 \sim \mathcal{N}(0, I)$ is comparable in magnitude to the data target.
2. **Condition bundle.** v_θ is conditioned on $c_t = \text{concat}(X_{t-1}, S_t, M_t, I)$, matching the StormCast teacher’s conditioning interface exactly. No extra VAE encoder is inserted, unlike the original FlowCast which works in latent space.
3. **Time rescaling.** The flow time $t \in [0, 1]$ is multiplied by $\tau = 1000$ before entering the SongUNet time embedding, because the positional-embedding frequencies of the pretrained architecture were tuned for the EDM $\log \sigma$ range.
4. **Channel-weighted + spectral loss.** The pointwise MSE is the channel-weighted L2 norm $\|\cdot\|_{2,\beta}$ of Section 3.8, and an additional radial log-PSD regulariser is applied

to the implied clean residual $\hat{x}_1 = x_t + (1 - t) v_\theta$ on the qpepre channel. Applying the regulariser on $\hat{x}_1$ rather than on v_θ keeps the spectrum interpretable as a physical field's spectrum.



5. **Endpoint clamp.** Training samples $t \sim \mathcal{U}(t_\epsilon, 1 - t_\epsilon)$ with $t_\epsilon = 10^{-5}$. At $t = 0$ or $t = 1$ the conditional path collapses to a point mass and the Monte-Carlo loss gradient is unbounded; the clamp prevents numerical blow-ups at negligible bias cost.

The resulting training loop differs from I-CFM only in these five bookkeeping details; the underlying theorem (§C.11) and path derivation (§C.12) carry over unchanged, and the sampler (3.10) is the explicit Euler integrator of the straight-line ODE.